Sistema inteligente de planificación, optimización y toma de decisiones adversarias Modalidad individual. Desarrollo en Python y ejecución en el equipo personal del estudiante.



Universidad Tecnológica de Aguascalientes

Carrera: Licenciatura en Ingenierías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Ing. Froylán Alanís

Alumnos:

- Hernández Martínez Pamela Alejandra 240792

Grupo: ITIID 7-C

Fecha:12/11/26

Índice

1-Resumen ejecutivo 4

2-Introducción 4

3. Análisis 5

4. Planteamiento del problema y supuestos 5

4.1 Descripción del problema 5

4.2 Propósito del proyecto 6

4.3 Supuestos y restricciones 6

5. Definición del escenario y representación del problema 7

5.1 Representación de la red logística 7

5.2 Cuadrícula y movimientos permitidos 7

5.3 Punto de inicio, destino y puntos de entrega 7

5.4 Obstáculos y costos de desplazamiento 8

5.5 Recursos disponibles y restricciones 9

5.6 Competidor y disponibilidad de recursos 9

5.7 Configuración de escenarios, parámetros y semillas aleatorias 10

6. Arquitectura e integración del sistema 10

6.1 Organización del proyecto 10

6.2 Estructura de carpetas y archivos 10

6.3 Descripción de los módulos 13

6.4 Comunicación e intercambio de información entre módulos 14

6.5 Tecnologías y herramientas utilizadas 15

7. Desarrollo de los módulos 16

7.1 Módulo A: Búsqueda heurística (Greedy Best-First Search y A*) 16

7.2 Módulo B: Búsqueda local (Hill Climbing) 21

7.3 Módulo C: Metaheurísticas (Recocido Simulado y Algoritmo Genético) 25

7.4 Módulo D: Teoría de juegos (Matriz de pagos y equilibrio de Nash) 32

7.5 Módulo E: Búsqueda adversaria (Minimax y poda alfa-beta) 36

8. Integración del sistema (flujo principal) 40

8.1 Funcionamiento general de la integración 40

8.2 Interacción entre los módulos 40
8.3 Asignación de recursos y decisiones del competidor 41
8.4 Optimización del orden de entrega 41
8.5 Ejecución del sistema completo 41
8.6 Resultados del plan logístico final 42
9. Análisis comparativo y discusión de resultados 46
9.1 Comparación general de los algoritmos 46
9.2 Eficiencia y calidad de las soluciones 47
9.3 Ventajas y desventajas de los métodos utilizados 49
9.4 Interpretación de los resultados generales 50
10. Pruebas, limitaciones y amenazas a la validez 54
10.1 Pruebas realizadas 54
10.2 Validación de escenarios y casos límite 57
10.3 Detección de escenarios sin solución 58
10.4 Limitaciones del sistema 58
10.5 Amenazas a la validez de los resultados 59
11. Conclusiones y trabajo futuro 59
11.1 Conclusiones generales 59
11.2 Cumplimiento de los objetivos 60
12. Referencias 61
13. Anexos técnicos 61
13.1 Archivos de configuración JSON 61
13.2 Tablas completas de resultados experimentales 61
13.3 Gráficas complementarias 62
13.4 Archivos CSV 62
13.5 Evidencias de ejecución 62
13.6 Instrucciones adicionales de ejecución 62
14. Defensa del proyecto 63

1-Resumen ejecutivo

El presente proyecto consiste en el desarrollo de un sistema inteligente de planificación, optimización y toma de decisiones adversarias aplicado a un escenario de logística y entrega de pedidos. El objetivo principal es integrar diferentes algoritmos de inteligencia artificial para encontrar rutas, organizar los puntos de entrega, optimizar los recursos disponibles y tomar decisiones considerando la participación de un competidor.

Para desarrollar el sistema se utilizaron cinco módulos. El primero implementa los algoritmos Greedy Best-First Search y A* para encontrar rutas dentro de una cuadrícula de 12 × 12. El segundo utiliza Hill Climbing para mejorar el orden en que se visitan los puntos de entrega. El tercero integra Recocido Simulado y Algoritmo Genético para buscar mejores soluciones mediante diferentes estrategias de optimización. El cuarto utiliza teoría de juegos para analizar las decisiones de dos jugadores al reservar vehículos, mediante una matriz de pagos y el cálculo de equilibrios de Nash. Finalmente, el quinto implementa Minimax y poda Alfa-Beta para analizar la selección de puntos de entrega entre dos jugadores.

Se realizaron diferentes pruebas con escenarios configurados en archivos JSON, utilizando semillas y parámetros definidos para poder repetir los experimentos. Los resultados permitieron comparar el costo de las rutas, la cantidad de nodos explorados, los tiempos de ejecución y la calidad de las soluciones obtenidas por cada algoritmo.

Entre los resultados principales se encontró que A* obtuvo rutas de menor costo que Greedy en los tres escenarios comparados. En las pruebas de optimización con ocho entregas, el Recocido Simulado alcanzó el costo óptimo en 14 de 20 ejecuciones, mientras que el Algoritmo Genético lo consiguió en 11 de 20. Por otra parte, la poda Alfa-Beta redujo considerablemente la cantidad de nodos visitados respecto a Minimax, manteniendo el mismo valor de resultado.

Finalmente, se integraron los módulos en un flujo principal que utiliza los resultados de los algoritmos para generar un plan de entrega. Este plan incluye la reserva de vehículos, la asignación de entregas, el orden de visita, el costo de la ruta y el costo total, guardando la información obtenida en un archivo JSON.

2-Introducción

La inteligencia artificial permite desarrollar soluciones para diferentes problemas que requieren analizar información, encontrar alternativas y tomar decisiones. Uno de los campos donde puede aplicarse es la logística, ya que actividades como organizar entregas, encontrar rutas y administrar recursos pueden representar un reto cuando existen obstáculos, costos diferentes y recursos limitados.

En este proyecto se desarrolló un sistema inteligente que simula un escenario de entregas dentro de una cuadrícula de 12 × 12. En este espacio se establece un punto de inicio, un destino, diferentes puntos de entrega y obstáculos que impiden el paso. También se consideran costos de movimiento que pueden variar dependiendo de la casilla por la que se desplaza el sistema.

Para resolver este problema se implementaron diferentes algoritmos de inteligencia artificial. Algunos permiten encontrar rutas de menor costo, mientras que otros ayudan a mejorar el orden de las entregas. También se incorporaron métodos para analizar las decisiones de dos jugadores cuando los recursos disponibles no son suficientes y para representar situaciones en las que las decisiones de un competidor afectan el resultado.

El desarrollo se organizó en cinco módulos: búsqueda heurística, búsqueda local, metaheurísticas, teoría de juegos y búsqueda adversaria. Cada uno tiene una función específica, pero todos trabajan sobre la representación del escenario y sus datos para contribuir a la planificación general.

Por último, se realizaron pruebas con distintos escenarios y parámetros para comparar el funcionamiento de los algoritmos, revisar los resultados y conocer sus ventajas y limitaciones. De esta manera, el proyecto no solo busca obtener una solución, sino también analizar cómo se comportan diferentes métodos de inteligencia artificial ante un mismo problema.

3. Análisis

El desarrollo de este proyecto permitió analizar cómo diferentes algoritmos de Inteligencia Artificial pueden utilizarse para resolver problemas de planificación, optimización y toma de decisiones. A través de las pruebas realizadas se identificaron las ventajas y limitaciones de cada técnica, así como las diferencias en sus resultados. La integración de estos métodos permitió construir un sistema que busca optimizar las rutas de entrega y considerar las decisiones de un competidor frente a recursos limitados.

4. Planteamiento del problema y supuestos

4.1 Descripción del problema

En un escenario de entregas es necesario organizar los recorridos para llegar a diferentes puntos utilizando los recursos disponibles. Sin embargo, encontrar una ruta no siempre significa obtener la mejor solución, ya que pueden existir obstáculos, casillas con diferentes costos de desplazamiento y una cantidad limitada de vehículos.

El problema que aborda este proyecto consiste en planificar las entregas dentro de un espacio representado por una cuadrícula de 12 × 12, donde se deben conectar un punto de inicio con un destino y visitar diferentes puntos de entrega. El recorrido debe evitar los obstáculos y considerar los costos asociados a cada movimiento.

A esta situación se agrega la participación de un competidor, cuyas decisiones pueden afectar la disponibilidad y el costo de los recursos. En este caso, ambos jugadores pueden reservar vehículos, pero la cantidad disponible es limitada. Cuando las reservas superan los recursos existentes, se aplica un incremento en el costo y se realiza una distribución proporcional de los vehículos dentro del modelo de teoría de juegos.

También se considera que los dos jugadores toman decisiones sobre los puntos de entrega mediante un modelo de búsqueda adversaria. De esta manera, cada uno selecciona entregas por turnos, buscando obtener el mayor beneficio posible frente a las decisiones del otro jugador.

Debido a estas condiciones, el problema requiere utilizar diferentes métodos de inteligencia artificial que permitan encontrar rutas, mejorar el orden de visita, analizar la disponibilidad de recursos y evaluar las decisiones de los participantes.

4.2 Propósito del proyecto

Desarrollar un sistema inteligente en Python que integre algoritmos de búsqueda heurística, búsqueda local, metaheurísticas, teoría de juegos y búsqueda adversaria para resolver un problema de planificación logística.

Se busca comparar el funcionamiento de estos métodos mediante diferentes escenarios y experimentos, tomando en cuenta aspectos como el costo de las soluciones, los recursos utilizados, los tiempos de ejecución y la cantidad de operaciones realizadas.

También se pretende integrar los resultados de los diferentes módulos para generar un plan final que incluya la asignación de entregas, la reserva de vehículos y el orden de visita de los puntos seleccionados.

4.3 Supuestos y restricciones

Para desarrollar la simulación se establecieron las siguientes condiciones:

- El escenario está representado mediante una cuadrícula de 12 × 12 casillas.
- El punto de inicio se encuentra en la coordenada (0, 0) y el destino en (11, 11).
- Cada escenario debe contar con al menos tres puntos de entrega.
- Los obstáculos representan casillas por las que no se permite el desplazamiento.
- Los movimientos permitidos son hacia arriba, abajo, izquierda y derecha. No se permiten movimientos diagonales.
- Cada casilla transitable tiene un costo de desplazamiento mayor que cero. Estos costos pueden variar entre las diferentes casillas.
- Los puntos de inicio, destino y entrega deben encontrarse dentro de los límites del mapa y no pueden estar ubicados sobre obstáculos.
- El escenario debe ser alcanzable desde el punto de inicio. Si alguno de los puntos necesarios no puede alcanzarse, el sistema debe identificarlo y reportar que no existe solución.
- La cantidad de vehículos disponibles es de tres y el costo inicial por vehículo es de cinco unidades.
- Cada jugador puede elegir reservar entre uno y tres vehículos.
- Cuando la suma de las reservas supera la cantidad de vehículos disponibles, el costo por vehículo aumenta utilizando un factor de 1.5.
- Para el cálculo de los pagos en teoría de juegos se considera una ganancia de 20 unidades por cada vehículo utilizado. Este valor es un supuesto establecido para la simulación.
- Para el modelo de búsqueda adversaria se considera una ganancia de 30 unidades al calcular el valor de cada punto de entrega. Este valor también es un supuesto del modelo.

- Los jugadores se representan mediante dos participantes: MAX, que busca maximizar la utilidad, y MIN, que busca minimizarla.
- Se utiliza una semilla aleatoria para poder repetir las ejecuciones bajo las mismas condiciones.
- Los escenarios, movimientos, costos, recursos y parámetros se almacenan en archivos de configuración JSON para facilitar su modificación y validación.

5. Definición del escenario y representación del problema

Para representar el problema de planificación de entregas se utilizó una cuadrícula de 12 × 12 casillas. En ella se establecieron un punto de inicio, un destino, diferentes puntos de entrega, obstáculos y costos de desplazamiento. Esta representación permite que los algoritmos trabajen sobre un mismo escenario y busquen rutas considerando las condiciones del mapa.

Los escenarios se guardaron en archivos JSON dentro de la carpeta config, lo que permite modificar sus características sin tener que cambiar directamente el código de los algoritmos.

5.1 Representación de la red logística

La red logística se representa mediante una cuadrícula donde cada casilla corresponde a una ubicación por la que puede pasar el vehículo. Algunas casillas tienen obstáculos que impiden el paso, mientras que otras tienen diferentes costos de desplazamiento.

El sistema utiliza esta representación para calcular rutas entre el punto de inicio, las entregas y el destino final. Los algoritmos de búsqueda utilizan la información del escenario para determinar qué movimientos están permitidos y cuánto cuesta desplazarse entre las casillas.

5.2 Cuadrícula y movimientos permitidos

Todos los escenarios utilizan una cuadrícula de 12 filas por 12 columnas, con un total de 144 casillas.

Los movimientos permitidos son únicamente hacia arriba, abajo, izquierda y derecha. No se permiten movimientos diagonales. Esto significa que el vehículo avanza una casilla a la vez y debe respetar los límites del mapa.

Para calcular la distancia entre dos puntos se utiliza la distancia Manhattan, que considera los movimientos horizontales y verticales.

5.3 Punto de inicio, destino y puntos de entrega

En los escenarios se estableció el punto de inicio en la coordenada (0, 0) y el destino final en (11, 11). Las coordenadas representan primero la fila y después la columna.

Cada escenario cuenta con diferentes puntos de entrega. En los primeros tres escenarios se utilizan tres entregas, mientras que en el escenario 04 se utilizan ocho. Esto permite trabajar con diferentes cantidades de puntos y probar el comportamiento de los algoritmos.

Escenario	Punto de inicio	Destino	Número de entregas	Semilla
escenario_01	(0, 0)	(11, 11)	3	52
escenario_02	(0, 0)	(11, 11)	3	53
escenario_03	(0, 0)	(11, 11)	3	54
escenario_04	(0, 0)	(11, 11)	8	56

En el escenario 04, que se utiliza para la integración principal del sistema, los puntos de entrega son:

- (1, 3)
- (2, 9)
- (4, 1)
- (5, 8)
- (6, 4)
- (8, 10)
- (9, 6)
- (10, 2)

5.4 Obstáculos y costos de desplazamiento

Los obstáculos representan zonas por las que el vehículo no puede pasar. Cada escenario tiene una distribución diferente de obstáculos para modificar las rutas disponibles y probar distintas condiciones.

También se utiliza una matriz de costos de 12 × 12, en la que cada casilla tiene un valor de desplazamiento. Los costos son números positivos y pueden variar dependiendo de la ubicación. Por ejemplo, en algunos escenarios se utilizan valores de 1 a 3, mientras que en el escenario 02 existen casillas con costos de hasta 5.

Cuando el vehículo se mueve a una casilla, se suma el costo correspondiente a esa ubicación. Por esta razón, la ruta más corta en cantidad de movimientos no siempre representa la ruta de menor costo.

El sistema también cuenta con un escenario sin solución, en el que se colocaron obstáculos alrededor de uno de los puntos de entrega para comprobar que el programa pueda identificar cuando no existe una ruta disponible.

5.5 Recursos disponibles y restricciones

Los obstáculos representan zonas por las que el vehículo no puede pasar. Cada escenario tiene una distribución diferente de obstáculos para modificar las rutas disponibles y probar distintas condiciones.

También se utiliza una matriz de costos de 12 × 12, en la que cada casilla tiene un valor de desplazamiento. Los costos son números positivos y pueden variar dependiendo de la ubicación. Por ejemplo, en algunos escenarios se utilizan valores de 1 a 3, mientras que en el escenario 02 existen casillas con costos de hasta 5.

Cuando el vehículo se mueve a una casilla, se suma el costo correspondiente a esa ubicación. Por esta razón, la ruta más corta en cantidad de movimientos no siempre representa la ruta de menor costo.

El sistema también cuenta con un escenario sin solución, en el que se colocaron obstáculos alrededor de uno de los puntos de entrega para comprobar que el programa pueda identificar cuando no existe una ruta disponible.

Dentro de cada escenario se establecieron los siguientes recursos:

Recurso	Valor	Descripción
Transporte	3	Cantidad de vehículos disponibles.
Capacidad	10	Capacidad definida para el transporte.
Costo de transporte	5	Costo inicial por vehículo.

Estos recursos se utilizan principalmente en el módulo de teoría de juegos, donde los dos jugadores deben decidir cuántos vehículos reservar.

También se establecieron restricciones para los movimientos: el vehículo no puede salir de los límites de la cuadrícula, atravesar obstáculos ni utilizar casillas con costos iguales o menores a cero.

5.6 Competidor y disponibilidad de recursos

El escenario considera la participación de un competidor que utiliza el mismo recurso de transporte. Esto significa que las decisiones de ambos jugadores pueden afectar la disponibilidad de vehículos y el costo que deben pagar.

En la configuración se estableció que el recurso compartido es el transporte y que, cuando la cantidad de vehículos solicitados supera los disponibles, el costo aumenta mediante un factor de 1.5.

Esta situación se utiliza en el módulo de teoría de juegos para construir la matriz de pagos y analizar las estrategias de ambos jugadores. De esta manera, el sistema puede representar un escenario donde las decisiones de un competidor influyen en los recursos disponibles para realizar las entregas.

5.7 Configuración de escenarios, parámetros y semillas aleatorias

Los escenarios se almacenan en archivos JSON dentro de la carpeta config. Cada archivo contiene la información necesaria para ejecutar los algoritmos, como el tamaño de la cuadrícula, los puntos de entrega, los obstáculos, los costos, los movimientos permitidos, los recursos y la configuración del competidor.

Se utilizaron diferentes semillas aleatorias para que las ejecuciones puedan repetirse bajo las mismas condiciones. Por ejemplo, los escenarios 01, 02 y 03 utilizan las semillas 52, 53 y 54, respectivamente. El escenario 04 utiliza la semilla 56.

El programa también realiza validaciones al cargar un escenario. Comprueba que estén presentes los campos necesarios, que la cuadrícula y la matriz de costos tengan el tamaño esperado, que existan al menos tres entregas, que los puntos importantes estén dentro del mapa y que no se encuentren sobre obstáculos. También revisa que los costos sean positivos y que los puntos importantes sean alcanzables.

Si el archivo no existe, no contiene un JSON válido o algún dato no cumple las condiciones establecidas, el programa genera un error. Cuando no es posible llegar a uno de los puntos importantes, se identifica el escenario como un caso sin solución.

Los parámetros propios de cada algoritmo, como la temperatura inicial y el enfriamiento del recocido simulado, o el tamaño de población y las generaciones del algoritmo genético, se establecen en sus respectivos módulos de Python.

6. Arquitectura e integración del sistema

6.1 Organización del proyecto

El proyecto se organizó en diferentes carpetas y archivos de Python para mantener separados los algoritmos y facilitar su desarrollo, ejecución y mantenimiento. Cada módulo tiene una función específica, pero todos trabajan con la misma representación del escenario logístico.

La organización permite ejecutar los algoritmos por separado para analizar sus resultados y también ejecutar el archivo principal (main.py), que integra varios de ellos para generar un plan de entregas.

6.2 Estructura de carpetas y archivos

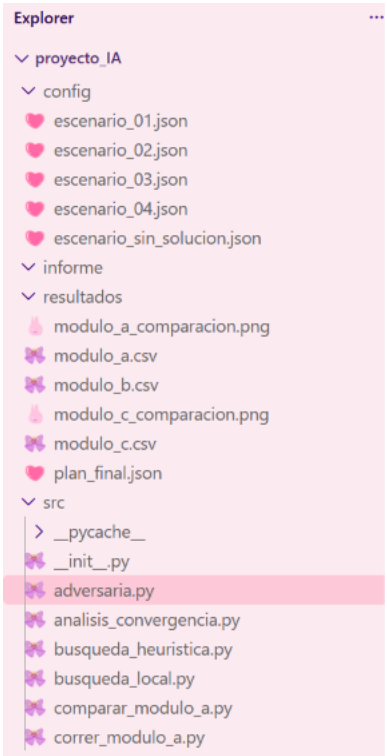
```
proyecto_IA/
│
├── config/
│   ├── escenario_01.json
│   ├── escenario_02.json
│   ├── escenario_03.json
│   ├── escenario_04.json
│   └── escenario_sin_solucion.json
```

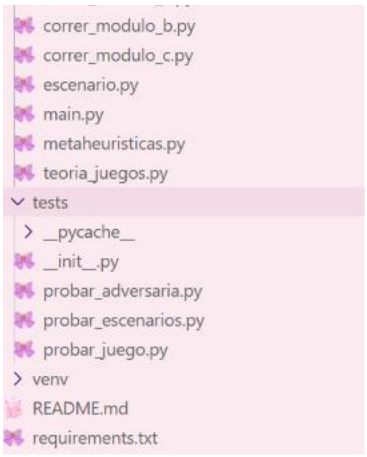
```
│
├── src/
│   ├── __init__.py
│   ├── escenario.py
│   ├── busqueda_heuristica.py
│   ├── correr_modulo_a.py
│   ├── comparar_modulo_a.py
│   ├── busqueda_local.py
│   ├── correr_modulo_b.py
│   ├── metaheuristicas.py
│   ├── correr_modulo_c.py
│   ├── analisis_convergencia.py
│   ├── teoria_juegos.py
│   ├── adversaria.py
│   └── main.py
│
├── tests/
│   ├── __init__.py
│   ├── probar_escenarios.py
│   ├── probar_juego.py
│   └── probar_adversaria.py
│
├── resultados/
│   ├── modulo_a.csv
│   ├── modulo_a_comparacion.png
│   ├── modulo_b.csv
│   ├── modulo_c.csv
│   ├── modulo_c_comparacion.png
│   └── plan_final.json
│
├── informe/
```

```
│
├── README.md
└── requirements.txt
```

Las carpetas y archivos principales tienen las siguientes funciones:

- **config:** contiene los escenarios en formato JSON, donde se definen el tamaño de la cuadrícula, los puntos de entrega, los obstáculos, los costos de movimiento, los recursos y las semillas aleatorias.
- **src:** contiene el código fuente y los algoritmos que forman parte del sistema.
- **tests:** contiene los archivos utilizados para comprobar el funcionamiento de los escenarios, la teoría de juegos y la búsqueda adversaria.
- **resultados:** almacena los archivos CSV con los resultados de los experimentos, las gráficas y el plan final generado por el sistema.
- **informe:** carpeta destinada a guardar el documento técnico del proyecto.
- **README.md:** explica la organización del proyecto, los requisitos y los comandos para ejecutar los diferentes módulos.
- **requirements.txt:** contiene las bibliotecas necesarias para ejecutar el proyecto.





6.3 Descripción de los módulos

El sistema se divide en cinco módulos principales, cada uno encargado de aplicar un algoritmo o método de inteligencia artificial.

Módulo A. Búsqueda heurística

Se encuentra principalmente en los archivos busqueda_heuristica.py, correr_modulo_a.py y comparar_modulo_a.py. Implementa los algoritmos Greedy Best-First Search y A* para encontrar rutas desde el punto de inicio hasta el destino, tomando en cuenta los obstáculos y los costos de movimiento.

También permite comparar los resultados de ambos algoritmos mediante el costo de la ruta, los nodos expandidos y el tiempo de ejecución.

Módulo B. Búsqueda local

Se encuentra en los archivos busqueda_local.py y correr_modulo_b.py. Implementa Hill Climbing para encontrar un orden de visita para los puntos de entrega.

Para esto utiliza una matriz de costos calculada mediante A* y genera nuevas soluciones intercambiando las posiciones de dos entregas. El algoritmo conserva los cambios que mejoran el costo de la ruta hasta que ya no encuentra una solución mejor.

Módulo C. Metaheurísticas

Se encuentra en metaheuristicas.py, correr_modulo_c.py y analisis_convergencia.py. Incluye dos métodos: Recocido Simulado y Algoritmo Genético.

Ambos buscan mejorar el orden de visita de los puntos de entrega, pero utilizan diferentes mecanismos. Recocido Simulado puede aceptar soluciones con un costo mayor dependiendo de la temperatura, mientras que el Algoritmo Genético trabaja con una población de soluciones que se modifican mediante selección, cruzamiento y mutación.

Módulo D. Teoría de juegos

Se encuentra en teoria_juegos.py y se comprueba mediante tests/probar_juego.py. Representa la interacción entre dos jugadores que deben decidir cuántos vehículos reservar.

El módulo construye una matriz de pagos, identifica las mejores respuestas, revisa si existen estrategias dominantes y encuentra los equilibrios de Nash. Con esto se analiza cómo las decisiones del competidor pueden afectar la disponibilidad y el costo del transporte.

Módulo E. Búsqueda adversaria

Se encuentra en adversaria.py y se comprueba mediante tests/probar_adversaria.py. Implementa Minimax y Alfa-Beta para representar la competencia por los puntos de entrega.

El módulo considera dos jugadores, MAX y MIN, que seleccionan entregas por turnos. También calcula el valor de las entregas y permite comparar los nodos visitados y las podas realizadas por ambos algoritmos.

6.4 Comunicación e intercambio de información entre módulos

Los módulos comparten información mediante los archivos de configuración JSON y las funciones definidas en los archivos de Python. De esta manera, no es necesario que cada algoritmo tenga una representación diferente del escenario.

El archivo escenario.py se encarga de cargar y validar la información de los escenarios. También proporciona funciones para consultar los puntos, verificar si una posición se puede recorrer, obtener los costos y conocer los movimientos permitidos.

A partir de esta información, los demás módulos pueden realizar sus cálculos. Por ejemplo, los algoritmos Greedy y A* utilizan los datos del escenario para calcular rutas, mientras que Hill Climbing y las metaheurísticas utilizan los costos obtenidos mediante A* para evaluar diferentes órdenes de entrega.

La integración principal se realiza en main.py, que sigue este proceso:

1. Carga el escenario 04 y establece la semilla aleatoria definida en su configuración.
2. Calcula los costos de desplazamiento entre el punto de inicio, las entregas y el destino mediante A*.
3. Construye la matriz de pagos de teoría de juegos y consulta los equilibrios de Nash para determinar la reserva de vehículos.
4. Utiliza Minimax para repartir los puntos de entrega entre el jugador y el competidor.
5. Genera diferentes órdenes de visita y compara los resultados de Hill Climbing y Recocido Simulado para seleccionar el mejor.
6. Calcula la ruta final mediante A* y obtiene el costo de la ruta y el costo total del transporte.
7. Guarda los resultados en el archivo resultados/plan_final.json.

De esta forma, los algoritmos no trabajan de manera completamente independiente, ya que sus resultados y cálculos se utilizan en diferentes partes del proceso para obtener un plan final de entregas.

Es importante mencionar que el Algoritmo Genético se utiliza en los experimentos del módulo C para comparar su desempeño con Recocido Simulado, pero no forma parte del flujo principal de main.py.

```
(venv) C:\Users\pamel\proyecto_IA>python -m src.main
Escenario: escenario_04 - semilla 56
Equilibrios de Nash: [(1, 2), (2, 1), (2, 2)]
Reservo 2 vehículos a 7.5 cada uno = 15.0
Mis entregas: [(6, 4), (4, 1), (5, 8), (8, 10)]
Entregas del competidor: [(1, 3), (9, 6), (2, 9), (10, 2)]
Mejor orden de visita: [(4, 1), (6, 4), (5, 8), (8, 10)] - costo 31

Costo de la ruta: 31
Costo de los vehículos: 15.0
Costo total del plan: 46.0
Plan guardado en resultados/plan_final.json

(venv) C:\Users\pamel\proyecto_IA>python -m src.main
Escenario: escenario_04 - semilla 56
Equilibrios de Nash: [(1, 2), (2, 1), (2, 2)]
Reservo 2 vehículos a 7.5 cada uno = 15.0
Mis entregas: [(6, 4), (4, 1), (5, 8), (8, 10)]
Entregas del competidor: [(1, 3), (9, 6), (2, 9), (10, 2)]
Mejor orden de visita: [(4, 1), (6, 4), (5, 8), (8, 10)] - costo 31

Costo de la ruta: 31
Costo de los vehículos: 15.0
Costo total del plan: 46.0
Plan guardado en resultados/plan_final.json
```

6.5 Tecnologías y herramientas utilizadas

Tecnología o herramienta	Uso dentro del proyecto
Python 3.11.9	Lenguaje utilizado para desarrollar los algoritmos y la integración del sistema.
JSON	Formato utilizado para almacenar la configuración de los escenarios.
CSV	Almacenamiento de los resultados obtenidos durante los experimentos.
NumPy	Biblioteca utilizada para trabajar con operaciones y estructuras numéricas.
Pandas	Biblioteca utilizada para organizar y analizar los resultados de los experimentos.
Matplotlib	Biblioteca utilizada para generar las gráficas comparativas.
Pytest	Herramienta incluida en los requisitos para realizar pruebas del proyecto.
Terminal de Windows	Utilizada para ejecutar los módulos y las pruebas mediante comandos de Python.

También se utilizó requirements.txt para especificar las bibliotecas necesarias y facilitar la instalación del entorno de trabajo. Por otra parte, el archivo README.md contiene las instrucciones para instalar y ejecutar el proyecto.

7. Desarrollo de los módulos

7.1 Módulo A: Búsqueda heurística (Greedy Best-First Search y A*)

7.1.1 Fundamento de los algoritmos

En este módulo se implementaron dos algoritmos de búsqueda heurística: Greedy Best-First Search y A*. Ambos se utilizaron para encontrar rutas dentro de una cuadrícula, considerando el punto de inicio, el destino, los obstáculos y los diferentes costos de desplazamiento.

Greedy Best-First Search selecciona el siguiente punto tomando en cuenta únicamente la distancia estimada que falta para llegar al destino. Para esto utiliza una función heurística (h(n)). Su objetivo es avanzar hacia las posiciones que parecen estar más cerca de la meta, sin considerar cuánto costó llegar hasta ellas.

Por otro lado, A* considera tanto el costo acumulado desde el inicio como el costo estimado que falta para llegar al destino. Para tomar sus decisiones utiliza la siguiente función:

$$f(n) = g(n) + h(n)$$

Donde:

- (g(n)): costo acumulado desde el punto de inicio hasta la posición actual.
- (h(n)): costo estimado desde la posición actual hasta el destino.
- (f(n)): costo total estimado de la ruta pasando por esa posición.

La principal diferencia es que Greedy se enfoca en acercarse al destino, mientras que A* también toma en cuenta el costo que ya se ha recorrido.

7.1.2 Heurística utilizada y justificación

Para ambos algoritmos se utilizó la distancia Manhattan como función heurística. Esta distancia permite calcular cuántos movimientos se necesitan como mínimo para llegar desde una posición hasta otra cuando únicamente se permite avanzar hacia arriba, abajo, izquierda y derecha.

Su fórmula es:

$$h(n) = |x_n - x_f| + |y_n - y_f|$$

Donde:

- (x_n,y_n): coordenadas de la posición actual.
- (x_f,y_f): coordenadas del destino.

Se eligió esta heurística porque el escenario utiliza movimientos en cuatro direcciones y no permite desplazamientos diagonales. Por esta razón, la distancia Manhattan representa una estimación adecuada de los pasos que faltan para llegar a la meta.

Por ejemplo, si la posición actual es (0,0) y el destino es (11,11), el cálculo es:

$$h(n) = |0 - 11| + |0 - 11| = 22$$

Esto significa que se necesitan como mínimo 22 movimientos para llegar al destino, sin considerar todavía los obstáculos ni los costos adicionales de las casillas.

7.1.3 Admisibilidad y condiciones de optimalidad

La heurística Manhattan es admisible en este proyecto porque nunca sobreestima el costo real que falta para llegar al destino.

Esto se debe a que cada movimiento tiene un costo mínimo de 1. Por lo tanto, el costo real de una ruta nunca puede ser menor que la cantidad mínima de movimientos calculada mediante la distancia Manhattan. Los obstáculos y las casillas con costos más altos pueden aumentar el costo de la ruta, pero no reducirlo.

Esta condición se representa de la siguiente manera:

$$h(n) \leq h^*(n)$$

Donde (h(n)) es el costo estimado y (h^*(n)) es el costo real mínimo para llegar al destino.

En las pruebas realizadas, la heurística calculada desde el inicio fue de 22 en los tres escenarios. Al compararla con los costos reales obtenidos mediante A*, se encontraron los siguientes resultados:

Escenario	Heurística Manhattan	Costo real de A*
01	22	22
02	22	29
03	22	22

Como se puede observar, la estimación nunca fue mayor que el costo real.

En cuanto a las condiciones de optimalidad, A* encuentra una ruta de costo mínimo cuando utiliza una heurística admisible y se cumplen las condiciones necesarias de búsqueda. En esta implementación también es importante que la heurística sea consistente, debido al uso de posiciones visitadas.

La consistencia significa que la estimación desde una posición no debe ser mayor que el costo de avanzar a una posición vecina más la estimación desde esa nueva posición:

$$h(n) \leq c(n, n') + h(n')$$

La distancia Manhattan cumple esta condición en el proyecto, ya que cada movimiento modifica la distancia estimada como máximo en una unidad y el costo de cada movimiento es de al menos 1.

Si se modificaran las reglas del escenario, por ejemplo, permitiendo movimientos diagonales o costos menores que 1, sería necesario revisar la heurística para conservar estas propiedades.

7.1.4 Implementación y funcionamiento

os algoritmos se implementaron en el archivo busqueda_heuristica.py. Para trabajar con los escenarios se utilizaron las funciones del archivo escenario.py, que permiten consultar las posiciones, verificar los movimientos permitidos, identificar los obstáculos y obtener los costos de las casillas.

El funcionamiento general de ambos algoritmos consiste en:

1. Obtener el punto de inicio y el destino del escenario.
2. Calcular la distancia Manhattan entre la posición actual y el destino.
3. Revisar los movimientos permitidos y descartar las posiciones que no se pueden recorrer.
4. Obtener el costo de desplazamiento de las casillas disponibles.
5. Seleccionar la siguiente posición de acuerdo con la función utilizada por cada algoritmo.
6. Repetir el proceso hasta encontrar el destino o determinar que no existe una ruta.

En Greedy Best-First Search se selecciona la posición con menor valor heurístico (h(n)). En A* se selecciona la posición con menor valor de (f(n)=g(n)+h(n)).

Los dos algoritmos registran información sobre la ruta encontrada, su costo, los nodos expandidos, el tiempo de ejecución y el resultado de la búsqueda. Esta información se guarda para poder comparar su comportamiento.

7.1.5 Escenarios de prueba y parámetros

Para evaluar el funcionamiento de los algoritmos se utilizaron tres escenarios diferentes, todos con una cuadrícula de 12 × 12 posiciones.

Escenario	Semilla	Puntos de entrega	Características
01	52	3	Cuadrícula con obstáculos y costos de movimiento de 1 a 3.
02	53	3	Mayor variación en los costos de movimiento, de 1 a 5, y diferentes obstáculos.
03	54	3	Incluye una barrera de obstáculos que obliga a buscar rutas alternativas.

En los tres escenarios se mantuvieron los movimientos en cuatro direcciones: arriba, abajo, izquierda y derecha. También se conservaron el punto de inicio (0,0) y el destino (11,11).

Los escenarios se almacenaron en archivos JSON dentro de la carpeta config. Para realizar las pruebas se ejecutaron los algoritmos sobre cada escenario y se registraron los resultados en resultados/modulo_a.csv.

Las semillas utilizadas permitieron mantener identificadas las configuraciones de los experimentos y facilitar su repetición.

7.1.6 Resultados obtenidos

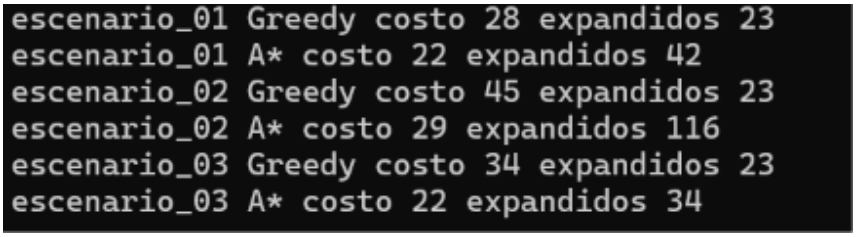
Después de ejecutar los dos algoritmos en los tres escenarios, se obtuvieron los siguientes resultados:

Escenario	Algoritmo	Costo de ruta	Nodos expandidos	Tiempo (ms)
01	Greedy	28	23	0.176
01	A*	22	42	0.787
02	Greedy	45	23	0.153
02	A*	29	116	2.389
03	Greedy	34	23	0.188
03	A*	22	34	0.349

Los resultados muestran que ambos algoritmos consiguieron encontrar una ruta en los tres escenarios.

En el escenario 01, Greedy obtuvo un costo de 28, mientras que A* encontró una ruta con costo de 22. En el escenario 02, la diferencia fue mayor: Greedy obtuvo un costo de 45 y A* uno de 29. Finalmente, en el escenario 03, Greedy registró un costo de 34 y A* uno de 22.

En cuanto al número de nodos expandidos, Greedy mantuvo 23 en los tres escenarios, mientras que A* necesitó expandir una mayor cantidad de nodos, especialmente en el escenario 02.



7.1.7 Comparación entre Greedy y A*

Al comparar los resultados se puede observar que Greedy y A* tienen comportamientos diferentes.

Greedy expandió menos nodos en los tres escenarios y también registró menores tiempos de ejecución. Sin embargo, sus rutas tuvieron un costo más alto que las encontradas por A*.

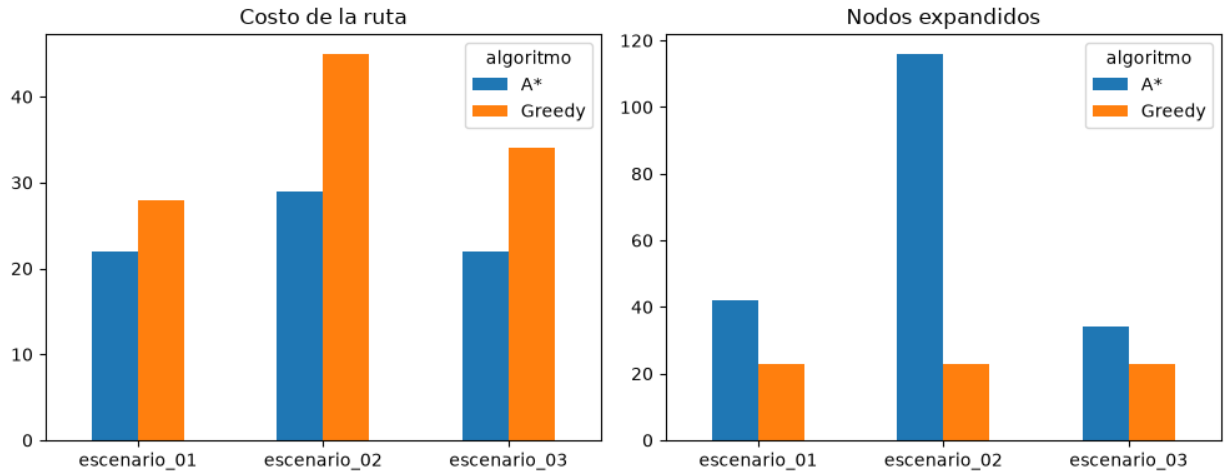
Por su parte, A* obtuvo rutas de menor costo en todos los escenarios, aunque necesitó revisar más posiciones para encontrarlas.

La diferencia porcentual entre los costos de las rutas fue la siguiente:

Escenario	Costo Greedy	Costo A*	Diferencia porcentual
01	28	22	27.3 %
02	45	29	55.2 %
03	34	22	54.5 %

Estos porcentajes muestran cuánto más costosa fue la ruta de Greedy respecto a la obtenida con A*.

También se puede observar que A* tuvo su mayor número de nodos expandidos en el escenario 02, con 116, debido a las características de ese escenario y a la distribución de sus costos y obstáculos.



7.1.8 Análisis de resultados

A partir de los experimentos se pudo observar que Greedy Best-First Search es una opción rápida, ya que en los tres escenarios expandió únicamente 23 nodos. Sin embargo, al tomar sus decisiones considerando solamente la distancia estimada hasta el destino, no siempre encuentra la ruta de menor costo.

En cambio, A* obtuvo mejores resultados en cuanto al costo de las rutas. Aunque necesitó expandir más nodos y tuvo tiempos de ejecución mayores en estas pruebas, logró encontrar las rutas más económicas en los tres escenarios.

Esto permite identificar una diferencia importante entre ambos algoritmos: Greedy busca avanzar hacia el destino de manera directa, mientras que A* toma en cuenta tanto el costo recorrido como el costo que falta.

Por lo tanto, si se busca reducir el costo de desplazamiento, A* resulta más conveniente para este proyecto. Greedy puede ser útil cuando se busca una respuesta rápida y se acepta que la ruta encontrada no necesariamente sea la más económica.

Finalmente, los resultados también permitieron comprobar que la distancia Manhattan fue adecuada para las condiciones de los escenarios utilizados, ya que mantuvo una estimación admisible y consistente.

7.2 Módulo B: Búsqueda local (Hill Climbing)

7.2.1 Fundamento del algoritmo

En este módulo se implementó el algoritmo Hill Climbing, también conocido como búsqueda de ascenso de colina. Este método permite encontrar soluciones mediante la mejora progresiva de una solución inicial.

Su funcionamiento consiste en comenzar con una solución y generar otras alternativas cercanas, llamadas soluciones vecinas. Después, compara sus costos y selecciona la mejor alternativa, siempre que represente una mejora respecto a la solución actual.

El proceso se repite hasta que ya no se encuentra una solución vecina que mejore el resultado.

Una característica de Hill Climbing es que solamente acepta cambios que mejoren la solución actual. Por esta razón, puede detenerse en un óptimo local, es decir, una solución que es mejor que todas las que tiene a su alrededor, pero que no necesariamente es la mejor de todas las soluciones posibles.

7.2.2 Representación de las soluciones candidatas

Para este proyecto, cada solución candidata representa un posible orden de visita para los ocho puntos de entrega del escenario 04.

La solución se representa mediante una lista de coordenadas. Cada coordenada corresponde a uno de los puntos que deben visitarse, y su posición dentro de la lista indica el orden en que se realizará la entrega.

Por ejemplo, una solución puede representarse de la siguiente manera:

```
[
    (4, 1),
    (6, 4),
    (5, 8),
    (8, 10),
    (1, 3),
    (9, 6),
    (2, 9),
    (10, 2)
]
```

Para calcular el costo de una solución se suman los desplazamientos desde el punto de inicio hasta cada entrega, siguiendo el orden establecido, y finalmente hasta el destino.

Los costos entre los puntos se obtienen mediante A*, por lo que se consideran los obstáculos y los diferentes costos de desplazamiento del escenario.

El objetivo del algoritmo es encontrar el orden de visita que tenga el menor costo total.

7.2.3 Vecindad y movimientos permitidos

Para generar soluciones vecinas se utilizó el intercambio de dos posiciones dentro de la lista de entregas.

Esto significa que se seleccionan dos puntos y se cambian de lugar, manteniendo los mismos ocho puntos de entrega. De esta manera, cada movimiento genera un nuevo orden que puede tener un costo diferente.

Como el escenario tiene ocho entregas, existen 28 intercambios posibles entre dos posiciones:

[
\frac{8(8-1)}{2}=28
]

Por ejemplo, si se intercambian las posiciones de las entregas (4,1) y (6,4), se obtiene un orden diferente al inicial.

El algoritmo revisa los 28 vecinos, compara sus costos y selecciona el que representa la mayor mejora. Si ninguno tiene un costo menor que el de la solución actual, el proceso termina.

Este tipo de vecindad permite modificar el orden de las entregas sin agregar puntos repetidos ni eliminar alguno de los existentes.

7.2.4 Implementación y funcionamiento

El algoritmo se implementó en el archivo busqueda_local.py, mientras que su ejecución y registro de resultados se realizan mediante correr_modulo_b.py.

Para calcular los costos se utiliza una matriz que contiene las distancias entre el punto de inicio, las entregas y el destino. Esta matriz se obtiene mediante el algoritmo A*.

El funcionamiento de Hill Climbing dentro del proyecto es el siguiente:

1. Se recibe un orden inicial de los puntos de entrega.
2. Se calcula el costo total de ese orden utilizando la matriz de costos.
3. Se generan los 28 vecinos mediante el intercambio de dos posiciones.
4. Se calcula el costo de cada vecino.
5. Se selecciona el vecino con menor costo, siempre que sea mejor que la solución actual.
6. Se repite el proceso hasta que ningún vecino consiga mejorar el costo.
7. Se registra la solución final, el número de pasos, las evaluaciones realizadas y el tiempo de ejecución.

En una de las ejecuciones de prueba, el costo inicial fue de 83 y se logró reducir a 64 después de dos pasos de mejora. Durante esa ejecución se realizaron 85 evaluaciones.

Estas evaluaciones corresponden a la solución inicial y a los vecinos revisados durante las tres rondas del proceso: dos rondas donde se encontró una mejora y una última donde ninguno de los 28 vecinos consiguió reducir el costo.

```
costo 64 pasos 2 evaluaciones 85
[(1, 3), (4, 1), (2, 9), (5, 8), (6, 4), (10, 2), (9, 6), (8, 10)]
```

7.2.5 Soluciones iniciales y parámetros

Para evaluar el comportamiento del algoritmo se realizaron 15 ejecuciones utilizando diferentes soluciones iniciales. Se utilizaron las semillas del 56 al 70 para identificar y reproducir cada ejecución.

Las pruebas se realizaron con el escenario 04, que contiene ocho puntos de entrega dentro de una cuadrícula de 12 × 12.

Los principales parámetros y características fueron los siguientes:

Parámetro	Valor o configuración
Escenario utilizado	escenario_04
Semillas	56 a 70
Número de entregas	8
Soluciones iniciales	15
Representación	Lista ordenada de coordenadas
Vecindad	Intercambio de dos entregas
Vecinos posibles por solución	28
Criterio de aceptación	Aceptar únicamente mejoras
Condición de terminación	No encontrar un vecino con menor costo

Se eligieron diferentes soluciones iniciales porque el resultado de Hill Climbing depende del punto desde el que comienza la búsqueda. Al realizar varias ejecuciones es posible observar qué tan frecuente es encontrar buenas soluciones y qué tanta variación existe entre los resultados.

7.2.6 Resultados obtenidos

Después de realizar las 15 ejecuciones se obtuvieron los siguientes resultados:

Indicador	Resultado
Número de ejecuciones	15
Mejor costo obtenido	59
Peor costo obtenido	72
Costo promedio	63.6
Desviación estándar	4.76
Resultados finales distintos	7
Ejecuciones que llegaron al óptimo global	5 de 15

Ejecuciones con vecino de igual costo al finalizar 1 de 15

Los costos finales obtenidos en las ejecuciones fueron:

59, 59, 59, 59, 59, 60, 60, 64, 65, 65, 66, 66, 69, 72 y 72.

El mejor costo encontrado fue de 59, mientras que el resultado más alto fue de 72. El promedio de las ejecuciones fue de 63.6 y la desviación estándar fue de 4.76.

También se comprobó el costo de las 40,320 permutaciones posibles de las ocho entregas, ya que (8!=40,320). Con esta revisión se confirmó que 59 corresponde al óptimo global para el problema evaluado.

Por lo tanto, Hill Climbing consiguió encontrar la mejor solución posible en cinco de las quince ejecuciones.

```
ejecucion,semilla,orden_inicial,costo_inicial,orden_final,costo_final,pasos,evaluaciones,tiempo_ms,vecinos_con_mismo_costo
1,56,"[(10, 2), (2, 9), (9, 6), (4, 1), (6, 4), (8, 10), (5, 8), (1, 3)]",111,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,8,253,0.49,0
2,57,"[(9, 6), (2, 9), (5, 8), (10, 2), (8, 10), (6, 4), (4, 1), (1, 3)]",103,"[(1, 3), (2, 9), (5, 8), (8, 10), (9, 6), (6, 4), (4, 1), (10, 2)]",66,3,113,0.208,0
3,58,"[(4, 1), (6, 4), (10, 2), (1, 3), (9, 6), (8, 10), (2, 9), (5, 8)]",77,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,4,141,0.375,0
4,59,"[(4, 1), (2, 9), (6, 4), (9, 6), (10, 2), (8, 10), (1, 3), (5, 8)]",93,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,4,141,0.26,0
5,60,"[(5, 8), (8, 10), (1, 3), (9, 6), (2, 9), (10, 2), (4, 1), (6, 4)]",112,"[(2, 9), (5, 8), (8, 10), (9, 6), (6, 4), (1, 3), (4, 1), (10, 2)]",72,5,169,0.307,0
6,61,"[(1, 3), (5, 8), (8, 10), (4, 1), (9, 6), (6, 4), (2, 9), (10, 2)]",95,"[(1, 3), (2, 9), (5, 8), (8, 10), (9, 6), (6, 4), (4, 1), (10, 2)]",66,3,113,0.22,0
7,62,"[(9, 6), (8, 10), (6, 4), (10, 2), (5, 8), (2, 9), (1, 3), (4, 1)]",93,"[(1, 3), (6, 4), (9, 6), (8, 10), (5, 8), (2, 9), (4, 1), (10, 2)]",69,5,169,0.308,0
8,63,"[(2, 9), (1, 3), (6, 4), (9, 6), (8, 10), (4, 1), (5, 8), (10, 2)]",96,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,4,141,0.259,0
9,64,"[(2, 9), (9, 6), (4, 1), (5, 8), (6, 4), (8, 10), (1, 3), (10, 2)]",111,"[(2, 9), (5, 8), (8, 10), (9, 6), (6, 4), (1, 3), (4, 1), (10, 2)]",72,3,113,0.222,0
10,65,"[(1, 3), (8, 10), (5, 8), (2, 9), (6, 4), (10, 2), (4, 1), (9, 6)]",81,"[(1, 3), (2, 9), (5, 8), (9, 6), (10, 2), (4, 1), (6, 4), (8, 10)]",65,4,141,0.256,0
11,66,"[(1, 3), (9, 6), (6, 4), (8, 10), (10, 2), (5, 8), (4, 1), (2, 9)]",98,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,4,141,0.258,0
12,67,"[(9, 6), (4, 1), (10, 2), (6, 4), (8, 10), (5, 8), (1, 3), (2, 9)]",97,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,4,141,0.344,0
13,68,"[(2, 9), (5, 8), (4, 1), (1, 3), (6, 4), (9, 6), (8, 10), (10, 2)]",83,"[(4, 1), (1, 3), (2, 9), (5, 8), (6, 4), (10, 2), (9, 6), (8, 10)]",64,4,141,0.482,1
14,69,"[(6, 4), (5, 8), (8, 10), (4, 1), (9, 6), (2, 9), (10, 2), (1, 3)]",116,"[(1, 3), (2, 9), (5, 8), (9, 6), (10, 2), (4, 1), (6, 4), (8, 10)]",65,7,225,0.584,0
15,70,"[(9, 6), (10, 2), (1, 3), (6, 4), (5, 8), (8, 10), (4, 1), (2, 9)]",100,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,6,197,0.372,0
```

7.2.7 Análisis de óptimos locales, mesetas y variabilidad

Los resultados permiten observar algunas limitaciones del algoritmo Hill Climbing, principalmente relacionadas con los óptimos locales y la dependencia de la solución inicial.

Óptimos locales

De las 15 ejecuciones se obtuvieron siete resultados finales distintos, con costos de 59, 60, 64, 65, 66, 69 y 72.

El algoritmo puede detenerse en cualquiera de estas soluciones cuando ninguno de sus 28 vecinos tiene un costo menor. Sin embargo, esto no significa que todas sean la mejor solución posible.

Por ejemplo, una solución con costo de 64 puede ser la mejor entre sus vecinos, pero todavía existe otra solución con costo de 59. Esto ocurre porque Hill Climbing no acepta movimientos que empeoren el resultado, por lo que no puede salir fácilmente de una zona donde ya no encuentra mejoras.

Mesetas

Una meseta ocurre cuando existen soluciones vecinas que tienen exactamente el mismo costo que la solución actual.

En las pruebas, una de las quince ejecuciones terminó con un vecino que tenía el mismo costo. Como el algoritmo solamente acepta soluciones que mejoran el resultado, no realiza el intercambio y termina la búsqueda.

Esto demuestra que el criterio de aceptación puede provocar que el algoritmo se detenga incluso cuando existen otras soluciones con el mismo costo.

Variabilidad de los resultados

La diferencia entre el mejor y el peor resultado también permite observar que el punto de partida influye en el comportamiento del algoritmo.

El mejor costo fue de 59, mientras que el peor fue de 72, lo que representa una diferencia aproximada del 22 % respecto al óptimo global. El promedio fue de 63.6, aproximadamente un 7.8 % por encima del mejor resultado.

La desviación estándar de 4.76 muestra que existió variación entre las soluciones encontradas en las diferentes ejecuciones.

Conclusión del módulo

En general, Hill Climbing permitió mejorar los órdenes iniciales de visita utilizando una cantidad relativamente pequeña de evaluaciones. Sin embargo, los resultados muestran que no siempre encuentra el óptimo global y que su desempeño depende de la solución inicial.

Aunque llegó al mejor costo en cinco de las quince ejecuciones, en las demás se detuvo en soluciones de mayor costo. Esto demuestra la importancia de utilizar diferentes soluciones iniciales y permite justificar el uso de otros métodos de búsqueda, como Recocido Simulado y el Algoritmo Genético, que se desarrollan en el siguiente módulo.

7.3 Módulo C: Metaheurísticas (Recocido Simulado y Algoritmo Genético)

7.3.1 Fundamento del Recocido Simulado

El Recocido Simulado es una metaheurística que permite buscar soluciones cercanas a la mejor posible mediante cambios progresivos en una solución inicial. Su funcionamiento está inspirado en el proceso de enfriamiento de los materiales, donde la temperatura disminuye poco a poco hasta alcanzar un estado más estable.

En este proyecto se utilizó para encontrar un mejor orden de visita de los puntos de entrega. A diferencia de Hill Climbing, este algoritmo no acepta únicamente los cambios que mejoran el resultado, ya que también puede aceptar temporalmente soluciones con un costo mayor.

Esto le permite explorar diferentes alternativas y evitar quedarse tan fácilmente en un óptimo local. Conforme disminuye la temperatura, se vuelve menos probable que acepte soluciones que empeoran el costo.

7.3.2 Regla de aceptación y enfriamiento

El algoritmo utiliza una regla probabilística para decidir si acepta una solución que tiene un costo mayor al de la solución actual.

La probabilidad de aceptación se calcula mediante la siguiente expresión:

$$
[
P = e^{-\Delta c/T}
]
$$

Donde:

- (P): probabilidad de aceptar una solución peor.
- (\Delta c): diferencia entre el costo de la nueva solución y el costo actual.
- (T): temperatura actual del algoritmo.

Cuando la nueva solución tiene un costo menor, se acepta directamente. Cuando tiene un costo mayor, se calcula su probabilidad de aceptación. Mientras la temperatura es alta, existe una mayor posibilidad de aceptar cambios que empeoran el resultado. Conforme baja la temperatura, esta posibilidad disminuye.

Para controlar el enfriamiento se utilizó un factor de 0.95, es decir, la temperatura se reduce al 95 % de su valor anterior en cada nivel.

De esta manera, el algoritmo comienza explorando diferentes soluciones y poco a poco se vuelve más selectivo, hasta llegar a la temperatura final establecida.

7.3.3 Fundamento del Algoritmo Genético

El Algoritmo Genético es una metaheurística inspirada en la selección natural. Su funcionamiento consiste en trabajar con un conjunto de soluciones, conocido como población, y generar nuevas alternativas mediante diferentes operaciones.

En este proyecto se utilizó para encontrar un orden de visita de los puntos de entrega con el menor costo posible.

Cada solución representa una posible organización de las entregas. El algoritmo evalúa los costos de las soluciones y utiliza las mejores como base para generar nuevas alternativas.

A lo largo de las generaciones, la población va cambiando mediante selección, cruzamiento y mutación. El objetivo es encontrar soluciones cada vez mejores sin depender de un solo orden inicial.

7.3.4 Población, selección, cruza y mutación

Para implementar el Algoritmo Genético se utilizaron los siguientes elementos:

- **Población:** se trabaja con un conjunto de 30 soluciones iniciales. Cada una representa un orden diferente de visita de los puntos de entrega.
- **Selección:** se utiliza selección por torneo, mediante la cual se comparan algunas soluciones y se eligen las que tienen mejores resultados para participar en la generación de nuevas soluciones.
- **Cruza:** se combinan dos soluciones para generar una nueva. Se conserva un segmento del primer orden y se completan las posiciones restantes con las entregas del segundo, evitando repetir puntos.
- **Mutación:** se utiliza una probabilidad de 0.1 para realizar un intercambio entre dos posiciones de una solución. Esto permite generar nuevas alternativas y mantener cierta variedad en la población.

También se conserva la mejor solución mediante elitismo, para evitar que se pierda una alternativa que ya había obtenido un buen resultado.

El proceso se repite durante un número establecido de generaciones, evaluando las nuevas soluciones y conservando las mejores alternativas encontradas.

7.3.5 Tratamiento de restricciones

Para ambos algoritmos se estableció que cada punto de entrega debe aparecer exactamente una vez dentro de la solución.

Esto se controla mediante la representación de las soluciones como permutaciones. En Recocido Simulado se intercambian dos posiciones del orden, mientras que en el Algoritmo Genético las operaciones de cruzamiento y mutación están diseñadas para evitar entregas repetidas o faltantes.

Por esta razón, no fue necesario agregar penalizaciones para las soluciones inválidas, ya que las operaciones utilizadas conservan un orden válido.

Los obstáculos y los costos de desplazamiento se consideran mediante una matriz de costos calculada con A*. De esta manera, los algoritmos de optimización utilizan los costos de las rutas entre los puntos de entrega sin tener que resolver nuevamente el recorrido dentro de la cuadrícula en cada evaluación.

Sin embargo, existe una limitación: en estos experimentos el costo que se optimiza corresponde únicamente al recorrido. Los recursos de transporte, la capacidad y el efecto del competidor no forman parte de la función de costo utilizada para evaluar los órdenes. Estos elementos se consideran en los módulos de teoría de juegos, búsqueda adversaria y en la integración final.

7.3.6 Parámetros y diseño de las ejecuciones

Para comparar los dos métodos se realizaron 20 ejecuciones independientes de cada algoritmo, utilizando el escenario 04, que contiene ocho puntos de entrega.

Los parámetros utilizados fueron los siguientes:

Parámetro	Recocido Simulado	Algoritmo Genético
Temperatura inicial	20	No aplica
Temperatura final	0.1	No aplica
Factor de enfriamiento	0.95	No aplica
Repeticiones por temperatura	10	No aplica
Tamaño de población	No aplica	30
Número de generaciones	No aplica	35
Probabilidad de mutación	No aplica	0.1
Tipo de selección	No aplica	Torneo
Ejecuciones	20	20

Las ejecuciones se identificaron mediante las semillas de la 56 a la 75. Esto permitió registrar los resultados y repetir los experimentos bajo las mismas condiciones.

Para evaluar el desempeño de los algoritmos se consideraron el mejor costo, el peor costo, el promedio, la desviación estándar, el número de evaluaciones y el tiempo de ejecución.

Los resultados se almacenaron en el archivo resultados/modulo_c.csv, mientras que la gráfica comparativa se guardó como resultados/modulo_c_comparacion.png.

7.3.7 Resultados obtenidos

Después de realizar las 20 ejecuciones de cada algoritmo, se obtuvieron los siguientes resultados:

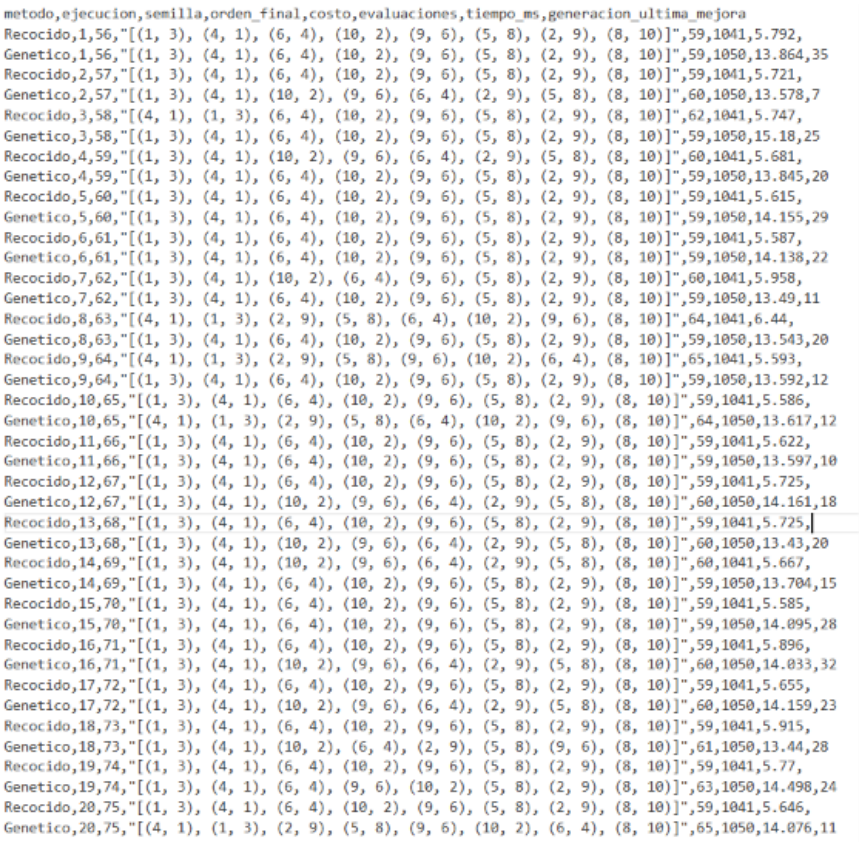
Indicador	Recocido Simulado	Algoritmo Genético
Mejor costo	59	59
Peor costo	65	65
Promedio	59.85	60.1
Desviación estándar	1.76	1.8
Ejecuciones que llegaron al óptimo	14 de 20	11 de 20
Evaluaciones	1,041	1,050
Tiempo promedio	5.75 ms	13.91 ms

Los dos algoritmos consiguieron encontrar una solución con costo de 59, que corresponde al mejor costo conocido para este problema.

Sin embargo, no todas las ejecuciones llegaron a ese resultado. Recocido Simulado alcanzó el costo de 59 en 14 de sus 20 ejecuciones, mientras que el Algoritmo Genético lo consiguió en 11.

También se observa que ambos métodos tuvieron un peor costo de 65, aunque el promedio de Recocido Simulado fue ligeramente menor.

En cuanto al tiempo de ejecución, Recocido Simulado necesitó en promedio 5.75 ms, mientras que el Algoritmo Genético registró 13.91 ms.



```
metodo,ejecucion,semilla,orden_final,costo,evaluaciones,tiempo_ms,generacion_ultima_mejora
Recocido,1,56,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.792,
Genetico,1,56,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.864,35
Recocido,2,57,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.721,
Genetico,2,57,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1050,13.578,7
Recocido,3,58,"[(4, 1), (1, 3), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",62,1041,5.747,
Genetico,3,58,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,15.18,25
Recocido,4,59,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1041,5.681,
Genetico,4,59,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.845,20
Recocido,5,60,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.615,
Genetico,5,60,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,14.155,29
Recocido,6,61,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.587,
Genetico,6,61,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,14.138,22
Recocido,7,62,"[(1, 3), (4, 1), (10, 2), (6, 4), (9, 6), (5, 8), (2, 9), (8, 10)]",60,1041,5.958,
Genetico,7,62,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.49,11
Recocido,8,63,"[(4, 1), (1, 3), (2, 9), (5, 8), (6, 4), (10, 2), (9, 6), (8, 10)]",64,1041,6.44,
Genetico,8,63,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.543,20
Recocido,9,64,"[(4, 1), (1, 3), (2, 9), (5, 8), (9, 6), (10, 2), (6, 4), (8, 10)]",65,1041,5.593,
Genetico,9,64,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.592,12
Recocido,10,65,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.586,
Genetico,10,65,"[(4, 1), (1, 3), (2, 9), (5, 8), (6, 4), (10, 2), (9, 6), (8, 10)]",64,1050,13.617,12
Recocido,11,66,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.622,
Genetico,11,66,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.597,10
Recocido,12,67,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.725,
Genetico,12,67,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1050,14.161,18
Recocido,13,68,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.725,
Genetico,13,68,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1050,13.43,20
Recocido,14,69,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1041,5.667,
Genetico,14,69,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,13.704,15
Recocido,15,70,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.585,
Genetico,15,70,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1050,14.095,28
Recocido,16,71,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.896,
Genetico,16,71,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1050,14.033,32
Recocido,17,72,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.655,
Genetico,17,72,"[(1, 3), (4, 1), (10, 2), (9, 6), (6, 4), (2, 9), (5, 8), (8, 10)]",60,1050,14.159,23
Recocido,18,73,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.915,
Genetico,18,73,"[(1, 3), (4, 1), (10, 2), (6, 4), (2, 9), (5, 8), (9, 6), (8, 10)]",61,1050,13.44,28
Recocido,19,74,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.77,
Genetico,19,74,"[(1, 3), (4, 1), (6, 4), (9, 6), (10, 2), (5, 8), (2, 9), (8, 10)]",63,1050,14.498,24
Recocido,20,75,"[(1, 3), (4, 1), (6, 4), (10, 2), (9, 6), (5, 8), (2, 9), (8, 10)]",59,1041,5.646,
Genetico,20,75,"[(4, 1), (1, 3), (2, 9), (5, 8), (9, 6), (10, 2), (6, 4), (8, 10)]",65,1050,14.076,11
```

7.3.8 Comparación entre metaheurísticas y Hill Climbing

Para conocer mejor el comportamiento de las metaheurísticas, también se compararon sus resultados con los obtenidos mediante Hill Climbing en el módulo B.

Indicador	Hill Climbing	Recocido Simulado	Algoritmo Genético
Mejor costo	59	59	59
Peor costo	72	65	65
Promedio	63.6	59.85	60.1
Desviación estándar	4.76	1.76	1.8
Llegó al óptimo	5 de 15	14 de 20	11 de 20
Evaluaciones aproximadas	85	1,041	1,050

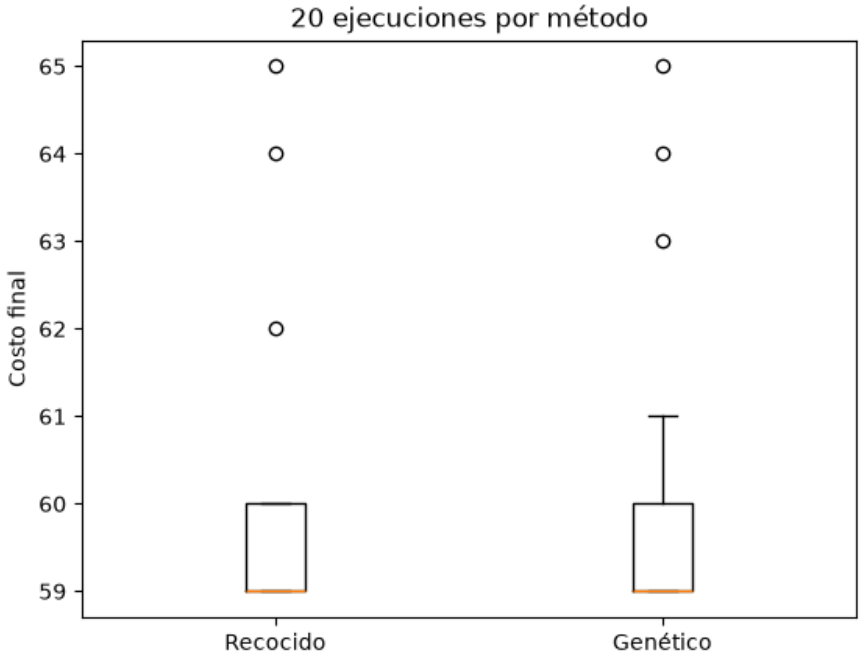
Los resultados muestran que las dos metaheurísticas tuvieron un mejor desempeño general que Hill Climbing. Aunque los tres métodos consiguieron encontrar el costo de 59, Recocido Simulado y el Algoritmo Genético llegaron a ese resultado en una mayor proporción de ejecuciones.

También se observa que Hill Climbing tuvo un peor costo de 72 y una desviación estándar de 4.76, mientras que las metaheurísticas tuvieron un peor costo de 65 y desviaciones cercanas a 1.8. Esto indica que sus resultados fueron menos variables en las pruebas realizadas.

Sin embargo, las metaheurísticas necesitaron muchas más evaluaciones. Mientras Hill Climbing realizó aproximadamente 85 evaluaciones por ejecución, Recocido Simulado y el Algoritmo Genético necesitaron alrededor de 1,040.

Entre las dos metaheurísticas, Recocido Simulado obtuvo resultados ligeramente mejores, ya que alcanzó el óptimo en más ejecuciones y tuvo un promedio de costo un poco menor. Sin embargo, la diferencia no es suficiente para afirmar que sea claramente superior, considerando que se realizaron solamente 20 ejecuciones por método.

El Algoritmo Genético también tuvo un tiempo promedio mayor, a pesar de realizar prácticamente el mismo número de evaluaciones. Esto se relaciona con las operaciones adicionales que realiza en cada generación, como la selección, el cruzamiento y la mutación.

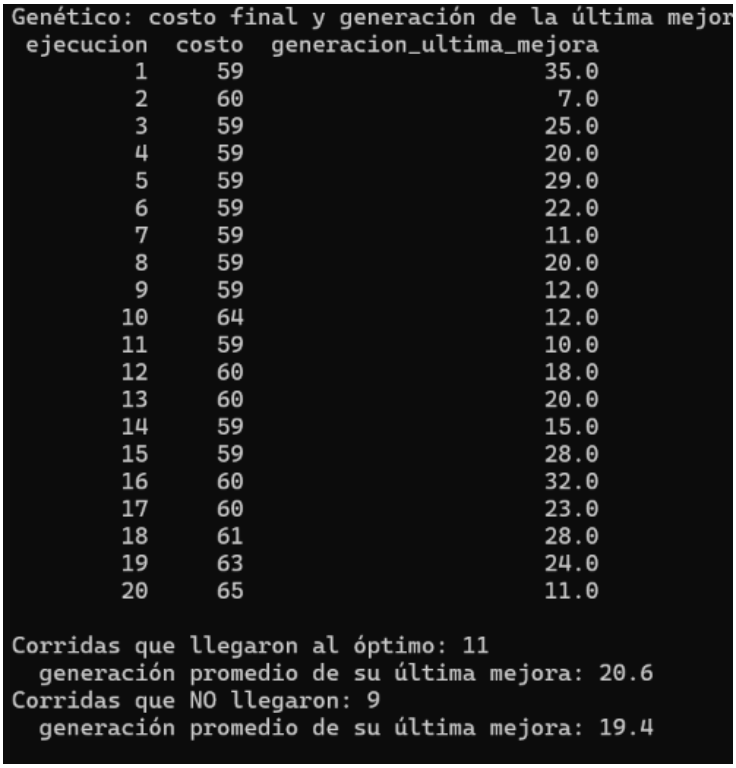


7.3.9 Análisis de convergencia prematura

Durante las ejecuciones del Algoritmo Genético se revisó el comportamiento de las generaciones para identificar posibles casos de convergencia prematura.

Este problema ocurre cuando el algoritmo deja de encontrar mejores soluciones demasiado pronto, aunque todavía existan alternativas que podrían mejorar el resultado.

Para analizarlo, se registró el costo final de cada ejecución y la generación en la que se encontró la última mejora.



```
Genético: costo final y generación de la última mejor
 ejecucion  costo  generacion_ultima_mejora
         1     59                     35.0
         2     60                      7.0
         3     59                     25.0
         4     59                     20.0
         5     59                     29.0
         6     59                     22.0
         7     59                     11.0
         8     59                     20.0
         9     59                     12.0
        10     64                     12.0
        11     59                     10.0
        12     60                     18.0
        13     60                     20.0
        14     59                     15.0
        15     59                     28.0
        16     60                     32.0
        17     60                     23.0
        18     61                     28.0
        19     63                     24.0
        20     65                     11.0

Corridas que llegaron al óptimo: 11
  generación promedio de su última mejora: 20.6
Corridas que NO llegaron: 9
  generación promedio de su última mejora: 19.4
```

Al revisar los resultados, se identificaron diferentes comportamientos:

- **Estancamiento temprano:** en las ejecuciones 2, 10 y 20, la última mejora ocurrió entre las generaciones 7 y 12. Después de ese momento, el algoritmo ya no encontró una solución mejor durante las generaciones restantes. Estos casos muestran indicios de convergencia prematura.
- **Falta de generaciones:** en las ejecuciones 16, 18 y 19, la última mejora ocurrió en las generaciones 32, 28 y 24, respectivamente. Esto indica que el límite de 35 generaciones pudo ser insuficiente para encontrar una solución mejor.
- **Comportamiento intermedio:** las ejecuciones 12, 13 y 17 tuvieron su última mejora en las generaciones 18, 20 y 23. En estos casos no se observa un estancamiento tan temprano, pero tampoco se alcanzó el costo óptimo.

También destaca la ejecución 1, que llegó al costo óptimo de 59 hasta la generación 35. Esto significa que el algoritmo todavía encontró una mejora en la última generación permitida.

Estos resultados muestran que no todas las ejecuciones que terminaron con un costo mayor al óptimo tuvieron el mismo comportamiento. En algunas se presentó un estancamiento temprano, mientras que en otras todavía se encontraban mejoras cuando se alcanzó el límite de generaciones.

Para intentar reducir este problema se podrían realizar algunos cambios, como aumentar el tamaño de la población, incrementar la probabilidad de mutación, permitir más generaciones o reemplazar parte de la población cuando pase mucho tiempo sin encontrar mejoras.

Con este análisis se puede observar que el Algoritmo Genético logró buenos resultados, pero su desempeño depende de los parámetros utilizados y de cómo evolucionan las soluciones durante cada ejecución.

7.3.10 Análisis de resultados

A partir de los experimentos se pudo observar que las dos metaheurísticas son útiles para encontrar mejores órdenes de visita de los puntos de entrega.

Recocido Simulado obtuvo el mejor promedio y alcanzó el costo óptimo en 14 de las 20 ejecuciones. Su regla de aceptación le permite explorar soluciones que pueden tener un costo mayor al principio, lo que ayuda a evitar quedarse en una solución local desde las primeras etapas.

Por su parte, el Algoritmo Genético también consiguió buenos resultados, aunque llegó al óptimo en 11 ejecuciones. Su funcionamiento permitió generar diferentes órdenes mediante la combinación y modificación de soluciones existentes. Sin embargo, algunas ejecuciones presentaron estancamiento antes de completar las generaciones disponibles.

Al compararlos con Hill Climbing, se observa que las metaheurísticas tuvieron resultados más constantes y llegaron al óptimo con mayor frecuencia, aunque necesitaron muchas más evaluaciones.

En conclusión, Recocido Simulado presentó los mejores resultados generales en estas pruebas, aunque la diferencia con el Algoritmo Genético fue pequeña. Ambos métodos representan alternativas para mejorar el orden de las entregas, especialmente cuando Hill Climbing puede quedarse detenido en un óptimo local.

7.4 Módulo D: Teoría de juegos (Matriz de pagos y equilibrio de Nash)

7.4.1 Descripción del juego y sus participantes

En este módulo se utiliza la teoría de juegos para representar la competencia entre dos jugadores que necesitan reservar vehículos para realizar sus entregas.

Los participantes son:

- **Jugador 1:** representa al sistema de planificación propio.
- **Jugador 2:** representa al competidor que también necesita utilizar los recursos de transporte.

Ambos jugadores toman sus decisiones de manera simultánea, sin conocer previamente la estrategia elegida por el otro. El objetivo es analizar cómo afecta la cantidad de vehículos reservados a las ganancias de cada participante, considerando que existe una cantidad limitada de recursos disponibles.

Para el modelo se establecieron los siguientes valores:

- Vehículos disponibles: 3.
- Costo base por vehículo: $5.
- Ganancia estimada por vehículo utilizado: $20.
- Factor de incremento por competencia: 1.5 cuando la demanda supera los recursos disponibles.

7.4.2 Estrategias de los jugadores

Cada jugador cuenta con tres estrategias posibles, que representan la cantidad de vehículos que desea reservar:

Estrategia	Vehículos solicitados
1	1 vehículo
2	2 vehículos
3	3 vehículos

Los dos participantes pueden elegir cualquiera de estas estrategias. Sin embargo, como solamente existen tres vehículos disponibles, cuando la suma de las reservas supera esa cantidad se presenta una situación de escasez.

En ese caso, el modelo incrementa el precio por vehículo utilizando el factor de competencia de 1.5 y distribuye los vehículos disponibles de manera proporcional a las cantidades solicitadas.

7.4.3 Construcción de la matriz de pagos

La matriz de pagos se construyó considerando las nueve combinaciones posibles entre las estrategias de ambos jugadores.

Para calcular los pagos se toma en cuenta la ganancia obtenida por los vehículos utilizados y el costo de los vehículos reservados. Cuando la demanda supera los tres vehículos disponibles, se aplica el incremento de precio y se realiza una distribución proporcional de los recursos.

La siguiente matriz muestra los pagos obtenidos. El primer valor corresponde al Jugador 1 y el segundo al Jugador 2.

Jugador 1 / Jugador 2	Estrategia 1	Estrategia 2	Estrategia 3
Estrategia 1	(15, 15)	(15, 30)	(7.5, 22.5)
Estrategia 2	(30, 15)	(15, 15)	(9, 13.5)
Estrategia 3	(22.5, 7.5)	(13.5, 9)	(7.5, 7.5)

Por ejemplo, cuando ambos jugadores eligen la estrategia 2, cada uno solicita dos vehículos. Como la demanda total es de cuatro vehículos y solamente existen tres disponibles, se incrementa el precio y se distribuyen los recursos proporcionalmente. Cada jugador recibe 1.5 vehículos y obtiene un pago de 15.

La matriz permite comparar los resultados de cada combinación y determinar qué decisiones resultan más convenientes para los participantes.

```
(venv) C:\Users\pamel\proyecto_IA>python -m tests.probar_juego
Matriz de pagos (jugador 1, jugador 2)
(1, 1) -> (15, 15)
(1, 2) -> (15, 30)
(1, 3) -> (7.5, 22.5)
(2, 1) -> (30, 15)
(2, 2) -> (15.0, 15.0)
(2, 3) -> (9.0, 13.5)
(3, 1) -> (22.5, 7.5)
(3, 2) -> (13.5, 9.0)
(3, 3) -> (7.5, 7.5)
```

7.4.4 Mejores respuestas

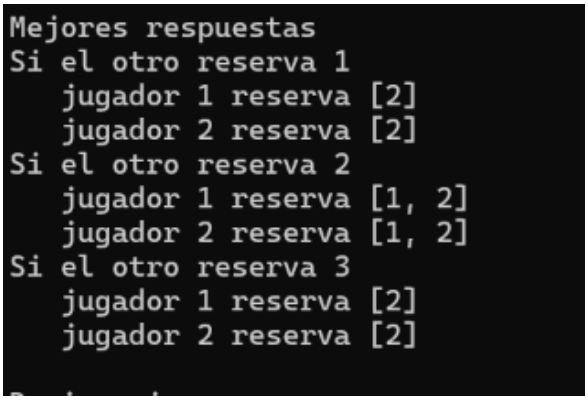
Las mejores respuestas representan la estrategia que proporciona el mayor pago a un jugador dependiendo de la decisión que haya tomado su oponente.

Al analizar la matriz de pagos se obtuvieron los siguientes resultados:

Estrategia del oponente	Mejor respuesta
Estrategia 1	Estrategia 2
Estrategia 2	Estrategia 1 o 2
Estrategia 3	Estrategia 2

Esto significa que, si el competidor reserva un vehículo, la mejor decisión es reservar dos. Si reserva dos, existen dos alternativas con el mismo pago: reservar uno o dos vehículos. Por último, si reserva tres, la mejor respuesta vuelve a ser reservar dos.

El mismo comportamiento se presenta para ambos jugadores, ya que el juego utiliza las mismas condiciones y reglas para los dos participantes.



7.4.5 Estrategias dominantes y dominancia

Al comparar los pagos de las tres estrategias se identificó que la estrategia 2 domina débilmente a la estrategia 1 y domina estrictamente a la estrategia 3.

Esto se debe a que:

- Frente a la estrategia 1 del oponente, elegir dos vehículos proporciona un pago mayor que elegir uno o tres.
- Frente a la estrategia 2 del oponente, elegir uno o dos vehículos produce el mismo pago, mientras que elegir tres genera un resultado menor.
- Frente a la estrategia 3 del oponente, elegir dos vehículos proporciona un pago mayor que las otras dos alternativas.

Por lo tanto, reservar dos vehículos es una decisión más conveniente que reservar tres en cualquiera de las situaciones analizadas. También resulta al menos igual de conveniente que reservar uno.

Es importante mencionar que la estrategia 2 no es estrictamente dominante frente a la estrategia 1, debido a que ambas producen el mismo pago cuando el oponente también elige la estrategia 2.

```
Dominancia
Jugador 1 - reservar 2 domina a reservar 1 (débil)
Jugador 1 - reservar 2 domina a reservar 3 (estricta)
Jugador 2 - reservar 2 domina a reservar 1 (débil)
Jugador 2 - reservar 2 domina a reservar 3 (estricta)
```

7.4.6 Equilibrios de Nash puros

Un equilibrio de Nash se presenta cuando ningún jugador puede mejorar su pago cambiando su estrategia de manera individual, mientras el otro mantiene la suya.

Después de analizar las mejores respuestas de ambos participantes, se identificaron tres equilibrios de Nash puros:

- (1, 2): el Jugador 1 reserva un vehículo y el Jugador 2 reserva dos.
- (2, 1): el Jugador 1 reserva dos vehículos y el Jugador 2 reserva uno.
- (2, 2): ambos jugadores reservan dos vehículos.

En los tres casos, ninguno de los participantes obtiene un beneficio al cambiar unilateralmente su decisión.

De estos resultados, el sistema utiliza el equilibrio (2, 2) cuando existen varios equilibrios posibles y ambos jugadores tienen la misma estrategia. En esta combinación, cada participante recibe un pago de 15.

Aunque ambos solicitan cuatro vehículos en total, solamente existen tres disponibles. Por esta razón, cada uno recibe 1.5 vehículos y paga un costo de 7.5.

```
Equilibrios de Nash: [(1, 2), (2, 1), (2, 2)]
```

7.4.7 Interpretación de los resultados en el contexto logístico

Los resultados muestran cómo la competencia por recursos limitados puede afectar las decisiones dentro de un problema logístico.

En este caso, reservar más vehículos no necesariamente representa una ventaja, ya que cuando ambos participantes solicitan una cantidad elevada se genera escasez y aumenta el costo del transporte.

La estrategia de reservar dos vehículos representa una alternativa conveniente, porque evita solicitar la cantidad máxima de recursos y permite obtener mejores resultados frente a las diferentes decisiones del competidor.

El equilibrio (2, 2), seleccionado por el sistema integrado, representa una situación en la que ambos participantes obtienen el mismo pago y ninguno tiene motivos para cambiar su decisión de manera individual.

Sin embargo, el modelo utiliza valores estimados para las ganancias y los costos, por lo que sus resultados representan una simulación de la competencia logística y no necesariamente el comportamiento de una empresa en una situación real.

Con este módulo se incorpora la teoría de juegos al proyecto, permitiendo analizar las decisiones de dos participantes, comparar sus posibles resultados e identificar combinaciones de estrategias estables mediante los equilibrios de Nash.

7.5 Módulo E: Búsqueda adversaria (Minimax y poda alfa-beta)

7.5.1 Fundamento de los algoritmos

En este módulo se implementan los algoritmos Minimax y poda alfa-beta para representar una situación en la que dos jugadores compiten por asignarse puntos de entrega.

Minimax es un algoritmo de búsqueda adversaria que analiza las posibles decisiones de dos participantes. Uno de ellos busca maximizar el resultado, mientras que el otro intenta minimizarlo. De esta manera, el algoritmo selecciona una decisión considerando las posibles respuestas del oponente.

Por otra parte, la poda alfa-beta es una optimización de Minimax que permite reducir la cantidad de estados que necesitan evaluarse. Para hacerlo, descarta ramas que ya no pueden modificar el resultado final de la búsqueda.

Ambos algoritmos se utilizan para analizar la asignación de entregas entre el sistema y su competidor.

7.5.2 Definición del subproblema adversario

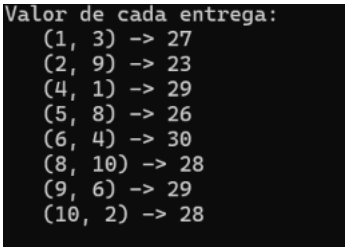
El subproblema consiste en decidir cómo se distribuyen los puntos de entrega entre dos participantes: el sistema propio y el competidor.

Cada punto tiene un valor que representa el beneficio estimado de atender esa entrega. Para calcularlo, se utiliza una ganancia base de 30 y se resta el costo adicional del recorrido, considerando la distancia desde el punto de inicio hasta la entrega y desde la entrega hasta el destino, en comparación con el recorrido directo.

Los valores obtenidos para los ocho puntos de entrega del escenario fueron:

Punto de entrega	Valor
(1, 3)	27
(2, 9)	23
(4, 1)	29
(5, 8)	26
(6, 4)	30
(8, 10)	28
(9, 6)	29
(10, 2)	28

El objetivo es que el sistema obtenga el mayor beneficio posible, mientras que el competidor intenta reducirlo mediante sus propias decisiones.



7.5.3 Representación de estados y acciones legales

Para representar el problema se utiliza un estado que contiene cuatro elementos:

- **Entregas restantes:** puntos que todavía no han sido asignados.
- **Puntuación MAX:** beneficio acumulado por el sistema propio.
- **Puntuación MIN:** beneficio acumulado por el competidor.
- **Turno:** indica qué jugador debe realizar la siguiente acción.

El jugador MAX representa al sistema propio y el jugador MIN representa al competidor.

Las acciones legales consisten en seleccionar cualquiera de los puntos de entrega que todavía se encuentren disponibles. Una vez seleccionado, el punto se elimina de las entregas restantes y su valor se suma a la puntuación del jugador que realizó el movimiento.

Al inicio del problema existen ocho entregas disponibles, por lo que el primer jugador cuenta con ocho acciones posibles.



7.5.4 Estados terminales y función de utilidad

Un estado terminal se presenta cuando ya no quedan entregas disponibles para asignar. En ese momento termina la búsqueda y se calcula el resultado obtenido por ambos participantes.

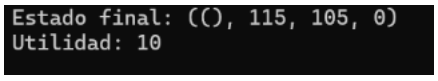
La función de utilidad utilizada es:

Utilidad = Puntuación MAX − Puntuación MIN

Esto permite determinar qué tan favorable fue el resultado para el sistema propio.

Si la utilidad es positiva, significa que MAX obtuvo una puntuación mayor que MIN. Si es negativa, significa que el competidor obtuvo un resultado superior. Si es cero, ambos participantes terminaron con la misma puntuación.

En una de las pruebas realizadas, el estado terminal obtuvo una puntuación de 115 para MAX y 105 para MIN, dando como resultado una utilidad de 10.



7.5.5 Implementación de Minimax

El algoritmo Minimax se implementó mediante una función recursiva que analiza las acciones disponibles en cada estado.

Cuando es el turno de MAX, el algoritmo selecciona el movimiento que produce la mayor utilidad. Cuando corresponde a MIN, selecciona el movimiento que produce la menor utilidad para MAX.

El proceso continúa hasta alcanzar un estado terminal o llegar a la profundidad máxima establecida.

En el escenario de prueba se evaluaron diferentes profundidades de búsqueda. Al utilizar las ocho entregas disponibles, el algoritmo pudo explorar todas las asignaciones posibles hasta llegar a los estados terminales.

7.5.6 Implementación de poda alfa-beta

La poda alfa-beta se implementó como una mejora del algoritmo Minimax. Su funcionamiento utiliza dos valores:

- **Alfa (α):** representa el mejor valor que MAX puede garantizar hasta el momento.
- **Beta (β):** representa el mejor valor que MIN puede garantizar hasta el momento.

Cuando se identifica que una rama ya no puede mejorar el resultado que se tiene registrado, se deja de explorar. Esto permite reducir la cantidad de nodos visitados sin modificar el resultado que obtendría Minimax.

También se realizaron pruebas con ordenamiento de acciones, evaluando primero las entregas con mayor valor. Esto permitió que se encontraran mejores límites durante la búsqueda y se descartaran más ramas.

7.5.7 Evaluación aproximada y profundidades de búsqueda

Para analizar el comportamiento de los algoritmos se utilizaron tres profundidades de búsqueda: 2, 4 y 8.

Cuando la búsqueda alcanza una profundidad limitada y todavía existen entregas sin asignar, se utiliza una evaluación aproximada. Esta evaluación ordena los valores de las entregas restantes de mayor a menor y los distribuye alternadamente entre MAX y MIN para estimar el resultado.

De esta manera, el algoritmo puede obtener una estimación sin necesidad de explorar todas las combinaciones posibles.

En las pruebas se observó que, para este escenario, las tres profundidades obtuvieron un valor de 6. Sin embargo, la cantidad de nodos evaluados y el tiempo de ejecución aumentaron considerablemente al utilizar una mayor profundidad.

```
profundidad 8 -> valor 6  nodos visitados 109601  tiempo 170.0 ms
profundidad 4 -> valor 6  nodos visitados 2081  tiempo 6.6 ms
profundidad 2 -> valor 6  nodos visitados 65  tiempo 0.3 ms
```

7.5.8 Resultados obtenidos

Se realizaron pruebas con Minimax y poda alfa-beta utilizando diferentes profundidades. Los resultados obtenidos fueron los siguientes:

Resultados de Minimax

Profundidad	Valor obtenido	Nodos visitados	Tiempo aproximado
2	6	65	0.3 ms

4	6	2,081	6.5 ms
8	6	109,601	180 ms

Resultados de poda alfa-beta

Profundidad	Ordenamiento	Valor obtenido	Nodos visitados	Nodos podados	Tiempo aproximado
2	Sin ordenar	6	65	0	0.3 ms
4	Sin ordenar	6	466	1,615	1.3 ms
4	Ordenadas	6	159	1,922	0.7 ms
8	Sin ordenar	6	9,393	100,208	15.9 ms
8	Ordenadas	6	1,657	107,944	4.4 ms

Los resultados muestran que ambos algoritmos obtuvieron el mismo valor de utilidad, aunque la cantidad de nodos visitados fue diferente.

```
Minimax contra alfa-beta
profundidad 8  ordenar False
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 109601  nodos alfa-beta 9393  podados 100208  podas 3721
   tiempo 15.9 ms
profundidad 8  ordenar True
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 109601  nodos alfa-beta 1657  podados 107944  podas 772
   tiempo 4.4 ms
profundidad 4  ordenar False
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 2081  nodos alfa-beta 466  podados 1615  podas 99
   tiempo 1.3 ms
profundidad 4  ordenar True
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 2081  nodos alfa-beta 159  podados 1922  podas 60
   tiempo 0.7 ms
```

7.5.9 Comparación de nodos visitados y podados

Al comparar los resultados se observa que Minimax necesita evaluar una mayor cantidad de nodos, especialmente cuando aumenta la profundidad de búsqueda.

En cambio, la poda alfa-beta permite descartar ramas que no pueden modificar el resultado final. Esto reduce considerablemente el trabajo necesario para encontrar la solución.

En la profundidad 8, Minimax visitó 109,601 nodos, mientras que alfa-beta sin ordenamiento visitó 9,393 y con ordenamiento solamente 1,657.

Esto representa una reducción aproximada del 91.4 % de los nodos visitados al utilizar alfa-beta sin ordenamiento y del 98.5 % al utilizar el ordenamiento de acciones.

En la profundidad 4 también se observa una mejora: Minimax visitó 2,081 nodos, mientras que alfa-beta con ordenamiento visitó 159.

El ordenamiento de acciones resultó importante porque permitió evaluar primero las entregas de mayor valor, facilitando la identificación de ramas que podían descartarse.

7.5.10 Análisis de resultados

Los resultados muestran que la poda alfa-beta mejora el rendimiento de Minimax al reducir la cantidad de nodos que necesita evaluar, sin cambiar el valor final obtenido.

La diferencia es más evidente cuando se utiliza una profundidad de búsqueda mayor. En la profundidad 8, Minimax necesitó aproximadamente 180 ms, mientras que alfa-beta con ordenamiento terminó en aproximadamente 4.4 ms.

También se observó que el ordenamiento de acciones mejora el funcionamiento de la poda, ya que permite descartar una mayor cantidad de ramas durante la búsqueda.

Por otra parte, las pruebas con diferentes profundidades muestran que una búsqueda más profunda requiere más tiempo y recursos computacionales. En este escenario, la profundidad 8 permitió explorar todas las asignaciones posibles, mientras que las profundidades 2 y 4 utilizaron una evaluación aproximada cuando todavía quedaban entregas disponibles.

En conclusión, Minimax permitió representar la competencia entre dos participantes y analizar sus posibles decisiones. La poda alfa-beta consiguió obtener el mismo resultado con una cantidad mucho menor de nodos visitados, por lo que resultó más eficiente para este problema.

8. Integración del sistema (flujo principal)

8.1 Funcionamiento general de la integración

La integración del sistema se realiza mediante el archivo main.py, que funciona como punto de entrada del proyecto y se encarga de ejecutar los diferentes módulos para generar un plan logístico.

El programa utiliza un escenario definido en un archivo JSON, donde se encuentran las dimensiones del mapa, el punto de inicio, el destino, las entregas, los obstáculos, los costos de movimiento, los recursos disponibles, la información del competidor y la semilla utilizada para las ejecuciones.

A partir de esta información, el sistema calcula las rutas entre los puntos, analiza las decisiones de los participantes, asigna las entregas y optimiza el orden en el que se deben visitar.

De esta manera, los módulos trabajan en conjunto para obtener una propuesta de planificación que considera tanto los recorridos como la competencia por los recursos disponibles.

8.2 Interacción entre los módulos

El archivo principal utiliza los resultados de los diferentes módulos para construir el plan logístico.

La interacción se realiza de la siguiente manera:

- **Módulo A. Búsqueda heurística:** utiliza el algoritmo A* para calcular los costos de recorrido entre el punto de inicio, las entregas y el destino.
- **Módulo B. Búsqueda local:** utiliza Hill Climbing para mejorar el orden en el que se visitan las entregas asignadas al sistema.
- **Módulo C. Metaheurísticas:** utiliza Recocido Simulado para buscar una mejor organización de las entregas y comparar su resultado con Hill Climbing.
- **Módulo D. Teoría de juegos:** construye la matriz de pagos y calcula los equilibrios de Nash para analizar las decisiones relacionadas con la reserva de vehículos.

- **Módulo E. Búsqueda adversaria:** utiliza Minimax para decidir cómo se distribuyen los puntos de entrega entre el sistema propio y el competidor.

Los módulos comparten la información del escenario y sus resultados se utilizan en diferentes etapas del proceso. No todos se ejecutan de la misma manera: por ejemplo, el Algoritmo Genético se utiliza en las pruebas del módulo C, pero no forma parte de la ejecución principal de main.py.

8.3 Asignación de recursos y decisiones del competidor

Una vez cargado el escenario, el sistema obtiene los recursos disponibles y construye la matriz de pagos del módulo de teoría de juegos.

Para la ejecución principal se establecen dos reservas de dos vehículos, una para cada participante. Esta decisión corresponde al equilibrio de Nash (2, 2), seleccionado cuando ambos jugadores utilizan la misma estrategia.

Como la cantidad total solicitada es de cuatro vehículos y solamente existen tres disponibles, se aplica el factor de competencia de 1.5. Como resultado, cada participante recibe 1.5 vehículos y el costo correspondiente es de 7.5 por participante.

Posteriormente, el módulo de búsqueda adversaria utiliza Minimax para determinar qué entregas corresponden al sistema propio y cuáles al competidor.

De esta manera, el sistema obtiene dos grupos de entregas, considerando los valores calculados para cada punto y las decisiones de ambos participantes.

8.4 Optimización del orden de entrega

Después de obtener las entregas asignadas al sistema propio, se busca determinar el orden más conveniente para visitarlas.

Para realizar este proceso, primero se construye una matriz de costos entre los puntos de entrega utilizando las rutas calculadas con A*.

Posteriormente, se generan diez soluciones iniciales con diferentes órdenes de visita. Cada una se evalúa mediante Hill Climbing y Recocido Simulado, utilizando el costo total del recorrido como criterio de comparación.

Ambos algoritmos buscan reducir la distancia y el costo del recorrido, considerando el punto de inicio, todas las entregas asignadas y el destino final.

Finalmente, el sistema compara los resultados obtenidos y selecciona el orden que presenta el menor costo.

Es importante mencionar que el Algoritmo Genético se utiliza en los experimentos del módulo C, pero no se ejecuta dentro de este proceso de integración.

8.5 Ejecución del sistema completo

Para ejecutar el proyecto integrado se utiliza el siguiente comando desde la carpeta principal del repositorio:

python -m src.main

Al iniciar, el programa realiza las siguientes actividades:

1. Carga el escenario configurado en el archivo JSON.

2. Establece la semilla para controlar la aleatoriedad de la ejecución.
3. Calcula los costos de recorrido entre los puntos mediante A*.
4. Construye la matriz de pagos y obtiene los equilibrios de Nash.
5. Establece las reservas de vehículos y calcula su costo.
6. Utiliza Minimax para asignar las entregas entre el sistema y el competidor.
7. Genera diferentes órdenes de visita para las entregas propias.
8. Compara Hill Climbing y Recocido Simulado para seleccionar el mejor orden.
9. Calcula la ruta completa desde el inicio, pasando por las entregas seleccionadas, hasta llegar al destino.
10. Guarda los resultados en el archivo resultados/plan_final.json.

Este proceso permite ejecutar los módulos principales desde un solo archivo, sin tener que iniciar cada uno de manera independiente.

```
(venv) C:\Users\pamel\proyecto_IA>python -m src.main
Escenario: escenario_04 - semilla 56
Equilibrios de Nash: [(1, 2), (2, 1), (2, 2)]
Reservo 2 vehículos a 7.5 cada uno = 15.0
Mis entregas: [(6, 4), (4, 1), (5, 8), (8, 10)]
Entregas del competidor: [(1, 3), (9, 6), (2, 9), (10, 2)]
Mejor orden de visita: [(4, 1), (6, 4), (5, 8), (8, 10)] - costo 31

Costo de la ruta: 31
Costo de los vehículos: 15.0
Costo total del plan: 46.0
Plan guardado en resultados/plan_final.json
```

8.6 Resultados del plan logístico final

Para comprobar el funcionamiento de la integración se utilizó el escenario 04, con semilla 56.

Los resultados obtenidos fueron los siguientes:

Configuración de recursos

Concepto	Resultado
Vehículos disponibles	3
Vehículos solicitados por cada participante	2
Vehículos asignados por participante	1.5
Costo por participante	$7.50
Costo total de los vehículos	$15.00

Asignación de entregas

Participante	Puntos de entrega
Sistema propio	(6, 4), (4, 1), (5, 8), (8, 10)

Competidor (1, 3), (9, 6), (2, 9), (10, 2)

Orden de visita seleccionado

El sistema obtuvo el siguiente orden para realizar sus entregas:

1. Inicio: (0, 0).
2. Primera entrega: (4, 1).
3. Segunda entrega: (6, 4).
4. Tercera entrega: (5, 8).
5. Cuarta entrega: (8, 10).
6. Destino final: (11, 11).

El costo del recorrido seleccionado fue de 31 unidades. Al agregar el costo de los vehículos, el costo total del plan logístico fue de 46 unidades.

Finalmente, el programa guardó la información del escenario, las reservas, la asignación de entregas, el orden de visita, los costos y la ruta completa en el archivo resultados/plan_final.json.

Estos resultados muestran que fue posible integrar los diferentes módulos para generar un plan logístico que considera la búsqueda de rutas, la competencia por recursos y la optimización del orden de entrega.

plan_final.json


```
{
  "escenario": "escenario_04",
  "semilla": 56,
  "vehiculos_reservados": 2,
  "precio_por_vehiculo": 7.5,
  "costo_vehiculos": 15.0,
  "entregas_mias": [
    [
      6,
      4
    ],
    [
      4,
      1
    ],
    [
      5,
      8
    ],
    [
      8,
      10
    ]
  ],
  "entregas_competidor": [
```

```
    [
      1,
      3
    ],
    [
      9,
      6
    ],
    [
      2,
      9
    ],
    [
      10,
      2
    ]
  ],
  "orden_visita": [
    [
      4,
      1
    ],
    [
      6,
      4
    ],
    [
      5,
      8
    ],
    [
      8,
      10
    ]
  ],
  "costo_ruta": 31,
  "casillas_en_ruta": 25,
  "costo_total_plan": 46.0,
  "ruta": [
    [
      0,
      0
    ],
    [
      1,
      0
    ],
    [
      2,
      0
    ],
```

```
[
  2,
  1
],
[
  3,
  1
],
[
  4,
  1
],
[
  4,
  2
],
[
  4,
  3
],
[
  4,
  4
],
[
  5,
  4
],
[
  6,
  4
],
[
  6,
  5
],
[
  6,
  6
],
[
  5,
  6
],
[
  5,
  7
],
[
  5,
  8
```

```
    ],
    [
      6,
      8
    ],
    [
      6,
      9
    ],
    [
      6,
      10
    ],
    [
      7,
      10
    ],
    [
      8,
      10
    ],
    [
      8,
      11
    ],
    [
      9,
      11
    ],
    [
      10,
      11
    ],
    [
      11,
      11
    ]
  ]
}
```


9. Análisis comparativo y discusión de resultados

9.1 Comparación general de los algoritmos

Durante el desarrollo del proyecto se utilizaron diferentes algoritmos de inteligencia artificial para resolver problemas relacionados con la planificación de rutas, la optimización del orden de entrega y la toma de decisiones frente a un competidor.

Cada algoritmo tiene una función diferente dentro del sistema. Por esta razón, su comparación se realiza considerando el objetivo que cumple, la calidad de sus resultados y los recursos que necesita para encontrar una solución.

Módulo	Algoritmos	Función principal
A. Búsqueda heurística	Greedy Best-First y A*	Encontrar rutas entre los puntos del escenario.
B. Búsqueda local	Hill Climbing	Mejorar el orden de visita de las entregas.
C. Metaheurísticas	Recocido Simulado y Algoritmo Genético	Buscar mejores órdenes de entrega mediante diferentes soluciones.
D. Teoría de juegos	Matriz de pagos y equilibrio de Nash	Analizar las decisiones de dos participantes al reservar vehículos.
E. Búsqueda adversaria	Minimax y poda alfa-beta	Analizar la asignación de entregas entre el sistema y el competidor.

En el módulo A, A* obtuvo rutas de menor costo que Greedy Best-First en los tres escenarios evaluados. Sin embargo, Greedy necesitó visitar menos nodos en las pruebas realizadas.

En el módulo B, Hill Climbing logró mejorar las soluciones iniciales, aunque algunas ejecuciones terminaron en soluciones locales que no correspondían al costo óptimo.

En el módulo C, Recocido Simulado y el Algoritmo Genético consiguieron mejores resultados generales que Hill Climbing en las pruebas realizadas. Ambos alcanzaron el costo óptimo de 59 en varias ejecuciones.

Por otra parte, el módulo D permitió identificar tres equilibrios de Nash puros, mientras que el módulo E mostró que la poda alfa-beta puede reducir considerablemente la cantidad de nodos evaluados por Minimax sin modificar el resultado.

9.2 Eficiencia y calidad de las soluciones

Para comparar la eficiencia y calidad de los algoritmos se consideraron los costos obtenidos, la cantidad de nodos visitados, las evaluaciones realizadas y los tiempos de ejecución, dependiendo del objetivo de cada módulo.

Módulo A: Búsqueda heurística

A* obtuvo los siguientes costos:

Escenario	Greedy Best-First	A*
01	28	22
02	45	29
03	34	22

En los tres escenarios, A* encontró rutas de menor costo. Greedy Best-First visitó 23 nodos en cada caso, mientras que A* visitó 42, 116 y 34 nodos, respectivamente.

Esto demuestra que A* necesitó explorar más nodos en algunos escenarios, pero consiguió mejores resultados en cuanto al costo de las rutas.

Módulo B y C: Optimización del orden de entrega

Para estos módulos se utilizó el mismo problema de optimización, considerando las ocho entregas y un costo óptimo de 59.

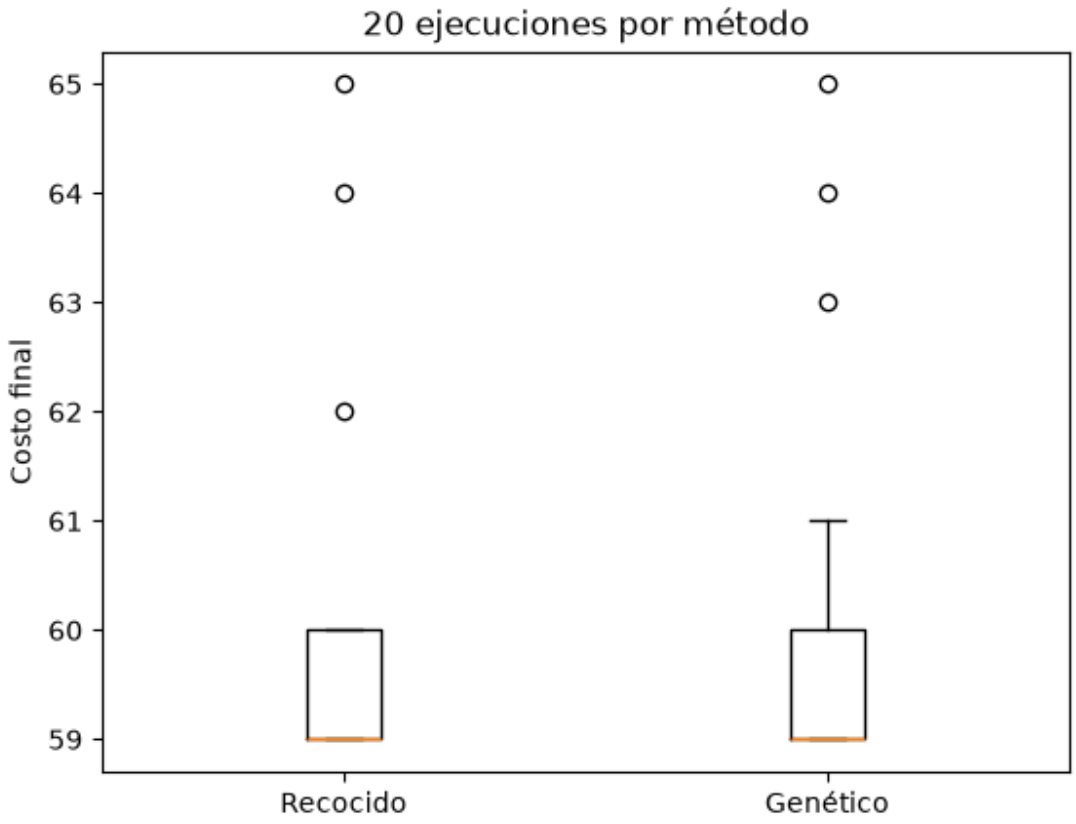
Algoritmo	Mejor costo	Peor costo	Promedio	Desviación estándar
Hill Climbing	59	72	63.60	4.76
Recocido Simulado	59	65	59.85	1.76
Algoritmo Genético	59	65	60.10	1.80

Hill Climbing alcanzó el costo óptimo en 5 de sus 15 ejecuciones, mientras que Recocido Simulado lo consiguió en 14 de 20 y el Algoritmo Genético en 11 de 20.

También se observó que Recocido Simulado y el Algoritmo Genético presentaron una menor variación en sus resultados. Esto indica que sus soluciones fueron más consistentes durante las pruebas realizadas.

En cuanto a las evaluaciones y tiempos, Recocido Simulado realizó aproximadamente 1,041 evaluaciones con un tiempo promedio de 5.75 ms. El Algoritmo Genético realizó aproximadamente 1,050 evaluaciones con un tiempo promedio de 13.91 ms.

Por lo tanto, Recocido Simulado presentó un resultado ligeramente mejor en promedio y necesitó menos tiempo que el Algoritmo Genético en estas pruebas.



Módulo D: Teoría de juegos

En este módulo se identificaron tres equilibrios de Nash puros: (1,2), (2,1) y (2,2).

El sistema seleccionó el equilibrio (2,2), en el que ambos participantes obtienen un pago de 15. Este resultado permite observar cómo la competencia por una cantidad limitada de vehículos influye en las decisiones de los jugadores.

Módulo E: Búsqueda adversaria

En las pruebas de Minimax y alfa-beta, ambos algoritmos obtuvieron el mismo valor de utilidad: 6.

Sin embargo, la cantidad de nodos visitados fue diferente. A profundidad 8, Minimax visitó 109,601 nodos, mientras que alfa-beta con ordenamiento de acciones visitó solamente 1,657.

Esto representa una reducción aproximada del 98.5 % en los nodos visitados, mostrando la ventaja de utilizar la poda alfa-beta.

9.3 Ventajas y desventajas de los métodos utilizados

Durante las pruebas se identificaron diferentes ventajas y limitaciones en los algoritmos implementados.

Algoritmo	Ventajas	Desventajas
Greedy Best-First	Visita pocos nodos y encuentra rutas rápidamente en los escenarios probados.	No garantiza encontrar la ruta de menor costo.
A*	Encuentra rutas óptimas con la heurística utilizada y considera el costo acumulado.	Puede necesitar explorar más nodos y utilizar más tiempo.
Hill Climbing	Es sencillo de implementar y mejora una solución inicial mediante movimientos locales.	Puede quedarse en óptimos locales y no encontrar la mejor solución global.
Recocido Simulado	Puede aceptar temporalmente soluciones peores y escapar de algunos óptimos locales.	Su resultado depende de la temperatura inicial, el enfriamiento y otros parámetros.
Algoritmo Genético	Trabaja con una población de soluciones y permite explorar diferentes órdenes de entrega.	Puede presentar estancamiento y requiere más tiempo por sus operaciones sobre la población.
Teoría de juegos	Permite analizar las decisiones de dos participantes y encontrar equilibrios de Nash.	Depende de los pagos y supuestos definidos para representar el problema.
Minimax	Analiza las posibles decisiones de ambos jugadores y considera las respuestas del competidor.	Puede requerir muchos nodos y tiempo cuando aumenta la profundidad.
Poda alfa-beta	Reduce la cantidad de nodos evaluados sin modificar el resultado de Minimax.	Su eficiencia depende del orden en el que se revisan las acciones.

En general, los resultados muestran que no existe un único algoritmo que sea el mejor para todas las situaciones. Cada uno resuelve una parte diferente del problema y tiene características que lo hacen más conveniente dependiendo del objetivo.

9.4 Interpretación de los resultados generales

Al analizar los resultados de los cinco módulos se puede observar que la integración de diferentes algoritmos permitió abordar el problema logístico desde distintas perspectivas.

La búsqueda heurística permitió calcular rutas entre los puntos de entrega, mientras que la búsqueda local y las metaheurísticas ayudaron a encontrar mejores órdenes de visita. Por su parte, la teoría de juegos permitió analizar la competencia por los vehículos disponibles y la búsqueda adversaria permitió distribuir las entregas entre los dos participantes.

Uno de los resultados más importantes fue la comparación entre Hill Climbing, Recocido Simulado y el Algoritmo Genético. Aunque los tres consiguieron alcanzar el costo óptimo de 59, Recocido Simulado presentó el mejor promedio y una menor variación en sus resultados dentro de las pruebas realizadas.

También destacó la poda alfa-beta, ya que permitió reducir considerablemente los nodos visitados por Minimax. Esto demuestra que es posible mejorar la eficiencia de una búsqueda adversaria sin cambiar el resultado obtenido.

En la ejecución integrada se utilizó el escenario 04, con semilla 56. El sistema seleccionó cuatro entregas para el recorrido propio y asignó las otras cuatro al competidor. Después de comparar los órdenes de visita, obtuvo un recorrido con costo de 31 unidades y un costo total de 46 unidades al agregar el costo de los vehículos.

Es importante considerar que los resultados de los módulos B y C corresponden a la optimización de las ocho entregas, mientras que el costo de 31 unidades de la integración corresponde únicamente a las cuatro entregas asignadas al sistema propio. Por esta razón, ambos resultados no deben compararse directamente.

También existen algunas limitaciones en el modelo. Los valores de ganancias y costos utilizados en teoría de juegos y búsqueda adversaria son estimaciones establecidas para las pruebas. Asimismo, la capacidad de transporte está definida en el escenario, pero todavía no se utiliza como una restricción dentro de la asignación de entregas.

En conclusión, los resultados muestran que los algoritmos implementados cumplen diferentes funciones dentro del proyecto y que su combinación permite generar un plan logístico considerando rutas, costos, recursos disponibles y decisiones de un competidor. Las pruebas también permitieron identificar las ventajas y limitaciones de cada método, así como las posibilidades de mejorar el sistema en futuras versiones.

plan_final.json


```
{
  "escenario": "escenario_04",
  "semilla": 56,
  "vehiculos_reservados": 2,
  "precio_por_vehiculo": 7.5,
  "costo_vehiculos": 15.0,
  "entregas_mias": [
    [
      6,
      4
    ],
    [
      4,
      1
    ],
    [
      5,
      8
    ],
    [
      8,
      10
    ]
  ],
  "entregas_competidor": [
    [
      1,
      3
    ],
    [
```

```
      9,
      6
    ],
    [
      2,
      9
    ],
    [
      10,
      2
    ]
  ],
  "orden_visita": [
    [
      4,
      1
    ],
    [
      6,
      4
    ],
    [
      5,
      8
    ],
    [
      8,
      10
    ]
  ],
  "costo_ruta": 31,
  "casillas_en_ruta": 25,
  "costo_total_plan": 46.0,
  "ruta": [
    [
      0,
      0
    ],
    [
      1,
      0
    ],
    [
      2,
      0
    ],
    [
      2,
      1
    ],
    [
```

```
    3,
    1
  ],
  [
    4,
    1
  ],
  [
    4,
    2
  ],
  [
    4,
    3
  ],
  [
    4,
    4
  ],
  [
    5,
    4
  ],
  [
    6,
    4
  ],
  [
    6,
    5
  ],
  [
    6,
    6
  ],
  [
    5,
    6
  ],
  [
    5,
    7
  ],
  [
    5,
    8
  ],
  [
    6,
    8
  ],
```

```
    [
      6,
      9
    ],
    [
      6,
      10
    ],
    [
      7,
      10
    ],
    [
      8,
      10
    ],
    [
      8,
      11
    ],
    [
      9,
      11
    ],
    [
      10,
      11
    ],
    [
      11,
      11
    ]
  ]
}
```


10. Pruebas, limitaciones y amenazas a la validez

10.1 Pruebas realizadas

Durante el desarrollo del proyecto se realizaron diferentes pruebas para comprobar el funcionamiento de los módulos y verificar que los algoritmos generaran resultados correctos.

Las pruebas se organizaron de acuerdo con las funciones principales del sistema:

- **Búsqueda heurística:** se probaron los algoritmos Greedy Best-First y A* utilizando diferentes escenarios, comparando los costos de las rutas, los nodos visitados y los tiempos de ejecución.

```
(venv) C:\Users\pamel\proyecto_IA>python -m src.correr_modulo_a
escenario_01 Greedy costo 28 expandidos 23
escenario_01 A* costo 22 expandidos 42
escenario_02 Greedy costo 45 expandidos 23
escenario_02 A* costo 29 expandidos 116
escenario_03 Greedy costo 34 expandidos 23
escenario_03 A* costo 22 expandidos 34
```

- **Búsqueda local:** se realizaron 15 ejecuciones de Hill Climbing con diferentes soluciones iniciales para analizar los costos obtenidos y comprobar si el algoritmo encontraba el mejor orden de entrega.

```
(venv) C:\Users\pamel\proyecto_IA>python -m src.correr_modulo_b
Costos finales: [59, 59, 59, 59, 59, 60, 60, 64, 65, 65, 66, 66, 69, 72, 72]
Mejor: 59  Peor: 72
Promedio: 63.6
Desviación estándar: 4.76
Órdenes finales distintos (óptimos locales): 7
Veces que llegó al mejor costo: 5 de 15
Corridas que terminaron en meseta: 1
Óptimo global (todas las permutaciones): 59
```

- **Metaheurísticas:** se realizaron 20 ejecuciones de Recocido Simulado y 20 del Algoritmo Genético para comparar sus resultados, promedios, variación y tiempos de ejecución.

```
(venv) C:\Users\pamel\proyecto_IA>python -m src.correr_modulo_c
Óptimo global: 59
Recocido
  mejor costo: 59
  peor costo: 65
  promedio: 59.85
  desviación estándar: 1.76
  llegó al óptimo: 14 de 20
  evaluaciones: 1041
  tiempo promedio (ms): 5.72
Genético
  mejor costo: 59
  peor costo: 65
  promedio: 60.1
  desviación estándar: 1.8
  llegó al óptimo: 11 de 20
  evaluaciones: 1050
  tiempo promedio (ms): 13.76
```

- **Teoría de juegos:** se comprobó la construcción de la matriz de pagos, las mejores respuestas, las estrategias dominantes y los equilibrios de Nash.

```
Matriz de pagos (jugador 1, jugador 2)
(1, 1) -> (15, 15)
(1, 2) -> (15, 30)
(1, 3) -> (7.5, 22.5)
(2, 1) -> (30, 15)
(2, 2) -> (15.0, 15.0)
(2, 3) -> (9.0, 13.5)
(3, 1) -> (22.5, 7.5)
(3, 2) -> (13.5, 9.0)
(3, 3) -> (7.5, 7.5)

Mejores respuestas
Si el otro reserva 1
   jugador 1 reserva [2]
   jugador 2 reserva [2]
Si el otro reserva 2
   jugador 1 reserva [1, 2]
   jugador 2 reserva [1, 2]
Si el otro reserva 3
   jugador 1 reserva [2]
   jugador 2 reserva [2]

Dominancia
Jugador 1 - reservar 2 domina a reservar 1 (débil)
Jugador 1 - reservar 2 domina a reservar 3 (estricta)
Jugador 2 - reservar 2 domina a reservar 1 (débil)
Jugador 2 - reservar 2 domina a reservar 3 (estricta)
```

- **Búsqueda adversaria:** se realizaron pruebas de Minimax y poda alfa-beta con diferentes profundidades, verificando que ambos algoritmos obtuvieran el mismo valor de utilidad.

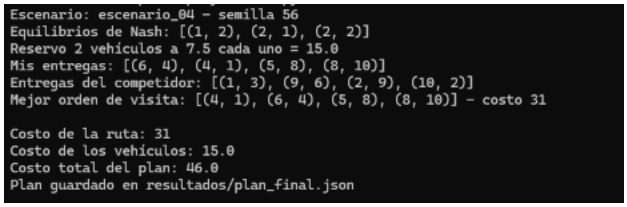
```
Valor de cada entrega:
   (1, 3) -> 27
   (2, 9) -> 23
   (4, 1) -> 29
   (5, 8) -> 26
   (6, 4) -> 30
   (8, 10) -> 28
   (9, 6) -> 29
   (10, 2) -> 28

Estado inicial: (((1, 3), (2, 9), (4, 1), (5, 8), (6, 4), (8, 10), (9, 6), (10, 2)), 0, 0, 0)
Acciones legales: 8
Estado final: ((), 115, 105, 0)
Utilidad: 10

profundidad 8 -> valor 6  nodos visitados 109601  tiempo 167.8 ms
profundidad 4 -> valor 6  nodos visitados 2081  tiempo 6.3 ms
profundidad 2 -> valor 6  nodos visitados 65  tiempo 0.3 ms

Minimax contra alfa-beta
profundidad 8  ordenar False
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 109601  nodos alfa-beta 9393  podados 100208  podas 3721
   tiempo 15.7 ms
profundidad 8  ordenar True
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 109601  nodos alfa-beta 1657  podados 107944  podas 772
   tiempo 3.7 ms
profundidad 4  ordenar False
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 2081  nodos alfa-beta 466  podados 1615  podas 99
   tiempo 1.4 ms
profundidad 4  ordenar True
   valor minimax 6  valor alfa-beta 6  iguales: True
   nodos minimax 2081  nodos alfa-beta 159  podados 1922  podas 60
   tiempo 0.6 ms
```

También se ejecutó el sistema completo mediante el comando python -m src.main, comprobando que los módulos principales trabajaran en conjunto y que se generara el archivo resultados/plan_final.json.



10.2 Validación de escenarios y casos límite

Para comprobar que el sistema pudiera trabajar con diferentes configuraciones, se crearon varios archivos JSON con distintos escenarios.

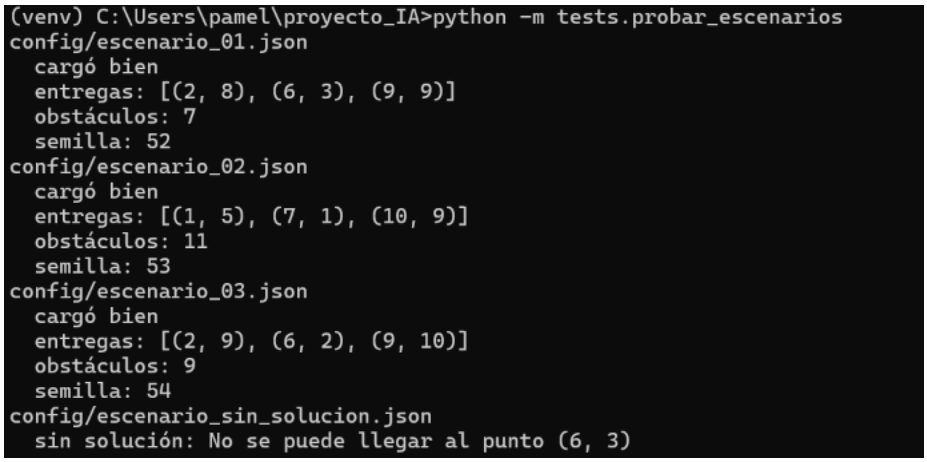
Se utilizaron cuatro escenarios funcionales y uno diseñado para comprobar qué sucede cuando no existe una solución.

Escenario	Semilla	Características
escenario_01.json	52	Escenario con tres entregas y obstáculos.
escenario_02.json	53	Escenario con tres entregas y diferentes costos de movimiento.
escenario_03.json	54	Escenario con tres entregas y una barrera de obstáculos.
escenario_04.json	56	Escenario con ocho entregas, utilizado en la integración.
escenario_sin_solucion.json	55	Escenario donde una entrega queda aislada por obstáculos.

Durante la validación se comprobó que los archivos contuvieran los datos necesarios, que el tamaño del mapa fuera de 12 × 12, que los costos fueran positivos y que los puntos importantes estuvieran dentro del mapa y no ocuparan posiciones bloqueadas.

También se verificó que existieran al menos tres entregas, que hubiera movimientos configurados y que la semilla fuera un número entero.

Estas comprobaciones permiten detectar errores en la configuración antes de ejecutar los algoritmos.



10.3 Detección de escenarios sin solución

Una de las pruebas realizadas consistió en crear un escenario en el que no fuera posible llegar a una de las entregas debido a la distribución de los obstáculos.

Para ello se utilizó el archivo escenario_sin_solucion.json, donde se colocaron obstáculos alrededor del punto (6, 3), impidiendo que se pudiera acceder a esa posición.

El sistema realiza una comprobación de conectividad antes de continuar con la ejecución. Cuando detecta que un punto importante no es alcanzable, genera el error SinSolucion.

Esta prueba permitió comprobar que el programa no continúa trabajando con un escenario que no cumple las condiciones necesarias para encontrar una ruta.

También se implementó el manejo de errores para detectar configuraciones inválidas, como archivos JSON incorrectos, campos obligatorios faltantes, dimensiones diferentes a las establecidas y costos de movimiento no válidos.

10.4 Limitaciones del sistema

Aunque el proyecto cumple con las funciones principales planteadas, durante su desarrollo se identificaron algunas limitaciones que pueden considerarse para futuras mejoras.

- **Capacidad de transporte:** el escenario contiene una capacidad de 10, pero actualmente no se utiliza como una restricción dentro de la asignación de entregas.
- **Costos estimados:** las ganancias y los costos utilizados en la teoría de juegos y la búsqueda adversaria son valores definidos para la simulación, por lo que no representan necesariamente cantidades de una operación logística real.
- **Búsqueda local:** Hill Climbing puede quedarse en óptimos locales y no siempre encuentra la mejor solución global.
- **Metaheurísticas:** los resultados dependen de los parámetros utilizados y de las soluciones iniciales. El Algoritmo Genético puede presentar indicios de estancamiento en algunas ejecuciones.
- **Búsqueda adversaria:** Minimax puede necesitar una gran cantidad de nodos cuando aumenta la profundidad. La poda alfa-beta reduce este problema, pero su eficiencia depende del orden de evaluación de las acciones.
- **Asignación de entregas:** el modelo de Minimax distribuye los puntos de acuerdo con sus valores estimados, pero no realiza una optimización conjunta de la capacidad y del costo total de las rutas de ambos participantes.
- **Integración:** el Algoritmo Genético se utiliza en las pruebas del módulo C, pero no forma parte de la ejecución principal del sistema.
- **Modelo de competencia:** la teoría de juegos utiliza una matriz de pagos con dos participantes y estrategias definidas previamente. No considera cambios en las decisiones durante varias rondas.

Estas limitaciones no impiden que el sistema funcione con los escenarios definidos, pero muestran algunos aspectos que podrían mejorarse para representar situaciones logísticas más completas.

10.5 Amenazas a la validez de los resultados

Las amenazas a la validez son situaciones que pueden influir en los resultados obtenidos o limitar la posibilidad de generalizarlos a otros problemas.

En este proyecto se identificaron las siguientes:

- **Dependencia de los escenarios:** los resultados se obtuvieron utilizando mapas y configuraciones específicas. Si cambian los obstáculos, las entregas o los costos, el comportamiento de los algoritmos también puede cambiar.
- **Uso de semillas:** las semillas permiten controlar la aleatoriedad y repetir las pruebas bajo condiciones similares. Sin embargo, una sola semilla no representa todos los resultados posibles.
- **Cantidad de ejecuciones:** aunque se realizaron 15 ejecuciones de Hill Climbing y 20 de cada metaheurística, una cantidad mayor permitiría obtener una comparación más amplia.
- **Parámetros de los algoritmos:** modificar la temperatura, el enfriamiento, la población, las generaciones o la probabilidad de mutación puede cambiar la calidad de las soluciones.
- **Tiempos de ejecución:** los tiempos registrados pueden variar dependiendo de la computadora, los procesos que se estén ejecutando y las condiciones del entorno.
- **Supuestos del modelo:** los pagos, ganancias y costos utilizados en los módulos D y E son estimaciones establecidas para realizar las simulaciones. Por lo tanto, los resultados dependen de esos valores.

En general, las pruebas permitieron comprobar el funcionamiento de los algoritmos y su integración dentro del proyecto. Sin embargo, los resultados deben interpretarse considerando los escenarios, parámetros y supuestos utilizados, ya que no garantizan el mismo comportamiento en cualquier situación logística.

11. Conclusiones y trabajo futuro

11.1 Conclusiones generales

Durante el desarrollo de este proyecto se logró construir un sistema inteligente que integra diferentes técnicas de Inteligencia Artificial para resolver un problema de planificación de entregas y toma de decisiones. A través de los módulos implementados fue posible trabajar con búsqueda heurística, búsqueda local, metaheurísticas, teoría de juegos y búsqueda adversaria.

Los resultados permitieron observar que cada algoritmo tiene un funcionamiento diferente y puede ser útil dependiendo del problema que se quiera resolver. En el módulo A, A* encontró rutas de menor costo que Greedy en los tres escenarios evaluados, aunque en algunos casos necesitó explorar más nodos. En el módulo B, Hill Climbing logró mejorar las soluciones iniciales, pero también presentó mínimos locales. Por otra parte, Recocido Simulado y el Algoritmo Genético consiguieron encontrar la solución óptima en varias de sus ejecuciones, aunque no siempre obtuvieron el mismo resultado.

En los módulos de teoría de juegos y búsqueda adversaria se pudo representar la competencia por los recursos disponibles y analizar cómo las decisiones de un jugador pueden afectar al otro.

También se comprobó que la poda alfa-beta permite reducir considerablemente la cantidad de nodos explorados por Minimax sin cambiar el valor de la solución.

En general, el proyecto permitió comprender que no existe un solo algoritmo que sea el mejor para todos los problemas. Cada técnica tiene ventajas y limitaciones, por lo que es importante elegirla de acuerdo con las características del escenario y los resultados que se esperan obtener.

11.2 Cumplimiento de los objetivos

Se cumplió con el objetivo general de desarrollar un sistema que integra diferentes algoritmos de Inteligencia Artificial para planificar rutas, optimizar el orden de las entregas y analizar decisiones frente a un competidor.

Respecto a los objetivos específicos, se lograron los siguientes resultados:

- Se implementaron los algoritmos Greedy Best-First Search y A* para encontrar rutas dentro de una cuadrícula de 12 × 12, considerando obstáculos y costos de movimiento.
- Se desarrolló Hill Climbing para mejorar el orden de las entregas y se evaluaron diferentes soluciones iniciales.
- Se implementaron Recocido Simulado y Algoritmo Genético, realizando 20 ejecuciones de cada uno para comparar sus resultados.
- Se construyó una matriz de pagos para representar las decisiones de dos jugadores, identificando mejores respuestas y equilibrios de Nash.
- Se implementaron Minimax y poda alfa-beta para analizar decisiones adversarias y comparar la cantidad de nodos explorados.
- Se integraron los módulos principales en un solo programa, utilizando un escenario compartido y guardando el resultado de la planificación en un archivo JSON.
- Se realizaron pruebas para comprobar la carga de escenarios y detectar aquellos en los que no existe una solución.

Aunque se alcanzaron los objetivos principales, también se identificaron algunas limitaciones. Por ejemplo, la capacidad de los vehículos está definida en la configuración, pero todavía no se aplica como una restricción dentro de la asignación de entregas. También, el Algoritmo Genético se evaluó en el módulo de metaheurísticas, pero no forma parte de la ejecución integrada principal.

11.3 Posibles mejoras y trabajo futuro

Como trabajo futuro, se podrían realizar diferentes mejoras para ampliar las capacidades del sistema:

- **Considerar la capacidad de los vehículos:** implementar restricciones para que la cantidad de entregas o la carga asignada no supere la capacidad disponible.
- **Mejorar la asignación de entregas:** integrar otros métodos de optimización para distribuir los puntos entre el usuario y el competidor de una manera más eficiente.
- **Integrar el Algoritmo Genético:** permitir que también participe en la planificación final y comparar sus resultados con Hill Climbing y Recocido Simulado.

- **Probar escenarios más grandes:** aumentar el tamaño de la cuadrícula, la cantidad de entregas, los obstáculos y los recursos disponibles para evaluar el comportamiento del sistema en problemas más complejos.
- **Mejorar la visualización:** desarrollar una interfaz gráfica que permita observar el mapa, las rutas, los obstáculos y los puntos de entrega de una manera más sencilla.
- **Optimizar los parámetros:** realizar más pruebas con diferentes temperaturas, tasas de enfriamiento, tamaños de población y generaciones para analizar si se pueden obtener mejores resultados.
- **Incorporar información en tiempo real:** considerar cambios en los costos, disponibilidad de vehículos u obstáculos que puedan aparecer durante la planificación.

Estas mejoras permitirían que el sistema fuera más completo y se acercara a un escenario de distribución real, donde las condiciones pueden cambiar y las decisiones deben adaptarse a los recursos disponibles.

12. Referencias

Universitat Politècnica de Catalunya. (2008). *Búsqueda informada y exploración* [Apuntes de clase, Inteligencia Artificial]. https://www.cs.upc.edu/~luigi/II/IA-2008-spring/2b-busqueda-informada-y-exploracion-%28es%29.pdf

13. Anexos técnicos

13.1 Archivos de configuración JSON

Se incluyen los archivos JSON utilizados para definir los escenarios de prueba. Cada archivo contiene información sobre el tamaño de la cuadrícula, punto de inicio, destino, entregas, obstáculos, costos de movimiento, recursos disponibles, competidor y semilla aleatoria.

Los archivos utilizados fueron:

- escenario_01.json
- escenario_02.json
- escenario_03.json
- escenario_04.json
- escenario_sin_solucion.json

Estos archivos se encuentran dentro de la carpeta config del proyecto.

13.2 Tablas completas de resultados experimentales

Se incluyen los resultados completos de las ejecuciones realizadas en los diferentes módulos. Estas tablas permiten consultar los valores obtenidos en las pruebas, sin limitarse únicamente a los promedios y comparaciones presentados en el informe.

Se consideran los resultados de búsqueda heurística, Hill Climbing, Recocido Simulado y Algoritmo Genético.

13.3 Gráficas complementarias

Se incluyen las gráficas generadas durante el análisis de los resultados experimentales, las cuales permiten observar las diferencias de rendimiento entre los algoritmos.

Entre los archivos se encuentran:

- modulo_a_comparacion.png
- modulo_c_comparacion.png

13.4 Archivos CSV

Se incluyen los archivos CSV generados por los módulos del sistema, los cuales contienen los resultados de las pruebas y permiten consultar los datos de manera organizada.

Los archivos son:

- modulo_a.csv
- modulo_b.csv
- modulo_c.csv

Estos archivos se encuentran dentro de la carpeta resultados.

13.5 Evidencias de ejecución

Se agregan capturas de pantalla de las ejecuciones realizadas desde la terminal, mostrando el funcionamiento de los módulos y las pruebas de validación.

Las evidencias incluyen:

- Ejecución de los módulos de búsqueda heurística, búsqueda local y metaheurísticas.
- Pruebas de teoría de juegos y búsqueda adversaria.
- Validación de los escenarios funcionales.
- Detección del escenario sin solución.
- Ejecución integrada del sistema y generación del archivo plan_final.json.

13.6 Instrucciones adicionales de ejecución

Para ejecutar el proyecto se requiere tener instalado Python 3.11.9 y las dependencias indicadas en el archivo requirements.txt.

Desde la terminal, se debe ingresar a la carpeta principal del proyecto y activar el entorno virtual. Posteriormente, instalar las dependencias con:

pip install -r requirements.txt

Para ejecutar el sistema integrado, utilizar:

python -m src.main

También es posible ejecutar los módulos y pruebas de manera individual:

python -m src.correr_modulo_a

python -m src.comparar_modulo_a

python -m src.correr_modulo_b

python -m src.correr_modulo_c

python -m src.analisis_convergencia

python -m tests.probar_juego

python -m tests.probar_adversaria

python -m tests.probar_escenarios

Al ejecutar el sistema integrado, los resultados de la planificación se guardan en el archivo resultados/plan_final.json.

14. Defensa del proyecto

El proyecto desarrollado demuestra cómo diferentes técnicas de Inteligencia Artificial pueden trabajar en conjunto para resolver un problema de planificación de entregas y toma de decisiones en un entorno donde existen recursos limitados y un competidor.

La elección de los algoritmos se realizó considerando las características de cada problema. Se utilizó A* para encontrar rutas de menor costo, Hill Climbing para mejorar el orden de las entregas, Recocido Simulado y Algoritmo Genético para explorar diferentes soluciones, teoría de juegos para analizar la competencia por los recursos y Minimax con poda alfa-beta para evaluar decisiones entre dos jugadores.

Los resultados obtenidos muestran que cada técnica aporta algo diferente al sistema. Por ejemplo, A* encontró rutas de menor costo que Greedy en los escenarios evaluados, las metaheurísticas consiguieron alcanzar la solución óptima en varias ejecuciones y la poda alfa-beta redujo considerablemente los nodos explorados sin modificar el resultado de Minimax.

Una de las principales fortalezas del proyecto es que no se limita a implementar los algoritmos por separado, sino que integra sus resultados en un proceso de planificación. El sistema considera un escenario compartido, analiza la disponibilidad de recursos, distribuye las entregas y determina una ruta final para completar el recorrido.

También se reconocen algunas limitaciones, como la falta de aplicación de la capacidad de los vehículos como restricción dentro de la asignación y la posibilidad de mejorar la integración de los algoritmos. Sin embargo, estas limitaciones representan oportunidades para continuar desarrollando el sistema.

Por lo anterior, se considera que el proyecto cumple con su propósito principal, ya que permite demostrar de manera práctica la aplicación, comparación e integración de diferentes técnicas de Inteligencia Artificial para resolver un problema que involucra optimización, competencia y toma de decisiones.